Documents sources complets

DictAI — Mode Réunion en direct

Source : docs/superpowers/specs/2026-09-24-meeting-mode.md
Rédaction et arbitrage : Astra, agent principal ; implémentation autorisée par l’utilisateur.

Mode Réunion en direct — plan d’implémentation

Source : docs/superpowers/plans/2026-09-24-meeting-mode.md
Auteur : Astra, agent principal de la conversation. Rédaction non déléguée. 24 septembre 2026.

DictAI — Mode Réunion en direct

Statut : spécification de développement, 24 septembre 2026. Rédaction et arbitrage : Astra, agent principal ; implémentation autorisée par l'utilisateur. Preuve native Android obtenue, intégration de la fonction en cours ; performances sur téléphone non mesurées.

Plan associé : plan d'implémentation détaillé. La rédaction et les mises à jour de ces deux documents restent la responsabilité de l'agent principal ; elles ne sont pas déléguées aux agents d'implémentation.

But

Permettre de participer à une réunion tout en suivant et en corrigeant une transcription qui attribue automatiquement les prises de parole. Conserver Nemotron pour reconnaître les mots. Activer une analyse des voix uniquement en mode Réunion.

Deux modes

Mode	Comportement
Dictée	Fonctionnement individuel actuel. Aucun chargement ni calcul de diarisation.
Réunion	Transcription progressive, attribution des passages aux intervenants, renommage et filtrage des voix pendant l'enregistrement.

Le mode est choisi avant l'enregistrement. Une nouvelle réunion crée un nouvel espace de profils. Une pause dans la même réunion conserve les profils et les noms. Une reprise après arrêt du processus nécessite une stratégie spécifique : les noms sauvegardés ne suffisent pas à réidentifier les voix après perte du cache acoustique.

Les noms et attributions restent sauvegardés avec le compte rendu de leur réunion. En revanche, le cache acoustique n'est pas réutilisé pour reconnaître les personnes dans une nouvelle réunion ; fermer une réunion permet de libérer ce cache et le modèle de diarisation.

Pastille et gestes — précision utilisateur du 24 septembre

Le choix du mode est accessible depuis le glissement vers le haut de la pastille, dans le menu qui donne déjà accès aux formats. Ajouter un choix explicite « Dictée / Réunion » en tête de ce menu. Les formats gardent leur sélection propre à Dictée ; choisir Réunion ne sélectionne pas un format de nettoyage. Le choix du mode reste possible depuis l’accueil et l’assistant. Une session en cours, même en pause, doit être terminée avant de changer de mode.

Cette règle vaut aussi pour la Dictée actuelle : sa pause conserve la session qui permet de reprendre. Un changement de mode affiche alors « Terminez la dictée avant de changer de mode. ». Une note simplement consultée ou retouchée, sans session d’enregistrement encore vivante, ne bloque pas le choix du mode ; elle est sauvegardée avant le prochain enregistrement.

En Réunion, conserver le dessin cursif, l’épaisseur du trait, les couleurs et l’enveloppe de la pastille. Les boucles forment une couronne circulaire qui tourne sur elle-même et donne un effet de spirale. Elles ne défilent plus horizontalement. Ce dessin indique le mode ; son mouvement indique une capture micro réellement active, y compris pendant un silence entre deux interventions. Les boucles restent fixes au repos, pendant la préparation, dès l’arrêt effectif du micro, en pause et après la fin. Le chargement reste indiqué séparément. Respecter le réglage Android qui désactive les animations et conserver un état textuel accessible.

État Réunion	Appui sur la pastille	Glissement vers le bas
Prête, sans session	Démarrer une nouvelle réunion	Aucune sauvegarde vide
Préparation	Aucun second démarrage	Aucune finalisation implicite
Écoute	Mettre en pause	Inviter à mettre en pause avant d’enregistrer
Mise en pause	Attendre la fin de la pause, sans reprise différée	Attendre la pause effective
Pause	Reprendre la même réunion, avec ses profils	Proposer d’enregistrer la transcription
Finalisation ou fermeture	Aucune commande concurrente	Aucune seconde sauvegarde

État Réunion	Appui sur la pastille	Glissement vers le bas
Terminée ou restaurée	Nouvelle réunion après sauvegarde de la note ouverte	Enregistrer les retouches de la note ouverte

La proposition affichée en pause porte « Enregistrer la transcription ? », avec « Enregistrer et terminer » et « Continuer la réunion ». La seconde commande, Retour ou une fermeture du dialogue laisse la réunion en pause ; seul un nouvel appui reprend l'écoute. Confirmer draine les derniers résultats, termine la session native et enregistre une note structurée avec les prénoms, attributions, retouches et images. Il s'agit de la transcription, pas d'un fichier audio. La confirmation de succès attend l'écriture durable. Une erreur conserve le document accessible et propose « Réessayer » ; elle ne relance pas le micro.

La sauvegarde automatique du brouillon continue pendant la réunion. Le geste vers le bas est la commande explicite de fin et d'enregistrement de la note. Le panneau propose les mêmes commandes accessibles, afin qu'aucune action ne dépende uniquement d'un geste. Les raccourcis de Dictée gardent leur comportement actuel ; le glissement vers le haut d'une dictée en pause conserve notamment son action existante d'insertion ou de réaffichage du texte.

Intervenants

Une note Réunion rouverte retrouve son document, ses intervenants et ses retouches, même si Dictée était sélectionné auparavant. Au repos, cette ouverture explicite sélectionne Réunion sans démarrer le microphone ; la couronne reste fixe. Les modèles absents n'empêchent pas de consulter ou corriger la note. Tant qu'un autre enregistrement possède sa session, y compris en pause ou pendant sa fermeture, l'ouverture d'une autre note est refusée en conservant le document courant. Une note rouverte ne récupère pas une mémoire vocale passée : un nouvel enregistrement crée une autre réunion après sauvegarde des dernières retouches.

- Créer « Personne 1 », puis « Personne 2 », etc., à mesure que des voix distinctes sont reconnues. Ne pas demander de présentation préalable.
- Conserver une identité interne stable, indépendante du nom affiché. Les profils sont propres à la réunion ; pas de reconnaissance d'identité entre réunions.
- Toucher un intervenant pour saisir son prénom. Le changement actualise ses étiquettes passées et futures, ainsi que les exports. Il ne remplace jamais un mot similaire dans le corps d'une phrase.
- Autoriser des prénoms identiques pour deux personnes différentes ; conserver un repère distinct dans le sélecteur.
- Autoriser la correction manuelle de l'intervenant d'un passage. Cette attribution devient prioritaire sur les révisions automatiques.
- Le modèle retenu propose huit canaux de locuteurs. Une voix ignorée reste suivie et occupe un canal. L'ignorer ne libère pas une neuvième place.
- Ne pas promettre la détection fiable d'une neuvième personne : au-delà de la capacité, le modèle peut confondre des voix. Ne jamais recycler volontairement l'identité d'une personne pour un nouveau locuteur.

Ignorer une voix

Décision de conception par défaut, réversible : masquer les interventions passées et futures de cette voix. L'interface indique « Masquer ses interventions dans cette réunion » et permet de tout réafficher.

Le profil reste dans la liste avec la mention « Ignoré » et une commande « Réafficher ». Les passages filtrés sont exclus du texte affiché et des exports ordinaires ; le texte interne reste disponible pour annuler le filtre. Le filtrage ne doit donc pas être présenté comme une suppression des données. Il ne nécessite pas de conserver un enregistrement audio permanent.

L'analyse continue de reconnaître la voix ignorée pour éviter de la recréer avec un autre numéro. Aucun gain de calcul n'est promis par ce filtrage. Si l'attribution reste incertaine, conserver le passage sous « Intervenant à confirmer » plutôt que le masquer arbitrairement.

Distinguer le filtrage des paroles attribuées et la séparation acoustique. Une voix forte qui couvre une autre personne peut empêcher la reconnaissance de certains mots. La diarisation seule ne restitue pas un signal propre et ne garantit pas la transcription de deux voix simultanées.

Direct et corrections

Le texte apparaît pendant la réunion, sans attendre l'arrêt de l'enregistrement. Les premiers mots peuvent précéder une attribution suffisamment stable ; utiliser alors une étiquette provisoire. Une révision de locuteur actualise les étiquettes en préservant les retouches du corps de texte.

« Instantané » signifie ici une transcription progressive avec un délai court à mesurer, jamais une promesse de délai nul. Mesurer séparément le délai des mots, celui de l'attribution et la réactivité des commandes. Les latences de tampon publiées par NVIDIA excluent le calcul et ne constituent pas des mesures sur Poco F7.

La correction manuelle reste disponible pendant l'enregistrement. Une révision ASR, un renommage, un changement d'attribution ou un filtre ne doit pas écraser les retouches. Préserver les suppressions, la composition clavier, la sélection, le point de lecture et les images. Les nouvelles paroles continuent d'arriver sans déplacer le curseur de correction.

Si une révision ne permet plus de retrouver avec certitude un passage corrigé, conserver le texte affiché et rendre la proposition reconnue consultable dans « Passages à vérifier ». Ne pas dupliquer tout l'énoncé ni élargir une attribution manuelle à des paroles nouvelles. Cette consultation reste accessible lorsqu'une voix est ignorée. Les images jointes restent à leur position dans le document, indépendamment du filtre vocal.

L'interface proposée comporte un choix « Dictée / Réunion », une liste compacte d'intervenants et un corps de transcription organisé par prises de parole. Les commandes « Renommer », « Ignorer » et « Réafficher » sont accessibles sans interrompre le micro. Une attribution incertaine reste visible et corrigeable.

Exemple de comportement

1. Personne 1 : « Nous pouvons commencer. »
2. Personne 2 : « Le rendez-vous est prévu mardi. »
3. L'utilisateur renomme Personne 1 en Sophie et Personne 2 en Karim.
4. Les deux étiquettes déjà affichées deviennent Sophie et Karim.
5. Sophie reprend la parole : son nouveau passage apparaît sous Sophie.
6. L'utilisateur corrige « mardi » en « jeudi » pendant que la transcription continue. Cette correction reste protégée.
7. Une troisième voix est reconnue ; l'utilisateur la marque « Ignoré ». Ses passages sont filtrés selon la portée choisie, sans interrompre Sophie ni Karim.

Architecture à respecter

Les données métier doivent conserver séparément : session, intervenants, segments audio horodatés, attributions automatiques, attributions manuelles, texte reconnu, retouches et état de filtrage. Le texte affiché est une projection de ces données ; il ne constitue pas la seule source de vérité.

Conserver la même horloge audio pour les mots et les activités des locuteurs. Les numéros de canaux du modèle ne doivent pas devenir directement des identifiants persistants sans gestion de la session. Ne pas relancer une diarisation indépendante sur chaque fragment en supposant que ses numéros correspondent aux précédents.

Ne pas demander à un modèle de nettoyage textuel de deviner les voix. Un éventuel nettoyage doit préserver les frontières de prises de parole, les noms et les retouches. Donner la priorité au micro et à la transcription ; ne pas charger automatiquement un modèle de nettoyage supplémentaire pour satisfaire ce mode.

Vérifier la capacité effective du modèle avant le démarrage : les exemples du SDK peuvent sélectionner par défaut l'ancien Sortformer à quatre voix. Utiliser le modèle à huit voix explicitement. Qualifier les dépendances natives pour éviter un conflit entre les versions de GGML déjà embarquées et celles du nouveau SDK.

En cas d'échec de la diarisation pendant une réunion, conserver les textes et les retouches, afficher l'indisponibilité de l'attribution et continuer la transcription si le runtime garantit encore un flux ASR valide. Si cette continuité n'est pas disponible, arrêter visiblement la capture et conserver le brouillon. Ne pas inventer de locuteur, ne pas réutiliser automatiquement un nom après réinitialisation, et ne pas basculer vers un service distant. Borner les files audio et la mémoire pour les longues réunions ; tout retard ou trou doit rester visible dans les mesures.

Point de départ vérifié, avant implémentation

- `DictationAsrEngine.kt` publie deux chaînes, texte confirmé et provisoire, sans segments de locuteurs.
- `TranscribeCppNative.kt` expose `full`, `committed` et `tentative`, sans horodatages de mots ni identifiants de locuteurs.
- `OverlayService.kt` et `EditableTranscript.kt` gèrent déjà l'édition pendant la dictée et la protection d'un préfixe modifié. Ajouter des noms dans cette chaîne plate ne suffirait pas à assurer renommage et filtrage fiables.
- `TranscriptNotes.kt` conserve principalement un texte et des images. La persistance structurée des réunions demande une extension compatible avec les notes existantes.

- Le moteur natif actuel est `handy-computer/transcribe.cpp`, révision `553f1099a2b3a5bc4421894be171f09960fc0f3a`. Le nouveau SDK NVIDIA est une intégration distincte à qualifier, pas une option déjà disponible dans le binaire actuel.
- La voie Réunion cible `NVIDIA/NeMo-Speech.cpp`, révision `97a15afa5caa9bce5baaa86c1184103877af4101`, et un pont JNI dédié vers son API C++. Cette API possède `set_interim_words`, `stable_speaker_time` et `refresh_speaker_tags`. Dans cette révision, l'API C ne branche pas `set_interim_words` : sa simple activation des options de diarisation ne suffit donc pas à garantir des mots attribués dans les résultats provisoires. La sonde doit vérifier cette différence.
- Le paquet Réunion utilisera les deux GGUF officiels compatibles avec ce runtime, dont les révisions et empreintes sont fixées dans le plan. Leur téléchargement total représente 848 560 480 octets, soit environ 849 Mo. Le GGUF Handy actuel reste réservé à la voie Dictée tant qu'une compatibilité n'est pas démontrée.
- Des modifications locales de gestion des images sont en cours. Toute implémentation devra être isolée et préserver ces modifications.
- ADB voit un émulateur Android arm64 ; aucun Poco F7 physique n'est connecté au moment de ce cadrage.

Validation avant de promettre le direct sur téléphone

1. Qualifier le modèle exact à huit locuteurs et son runtime natif : format, révision, licence, mémoire, API de streaming et stabilité des identifiants.
2. Exécuter une sonde Android arm64 alimentée au rythme réel d'un enregistrement français à trois voix. Vérifier que des étiquettes sortent avant la fin du fichier. Un traitement rapide après l'arrêt ne valide pas le besoin.
3. Mesurer Nemotron seul, puis Nemotron et diarisation ensemble, avec mêmes audio et réglages. Publier délais médians et au 95e percentile, retard accumulé, mémoire et erreurs d'attribution. Un débit moyen inférieur au temps réel ne suffit pas si les pointes rendent le direct inutilisable.
4. Valider sur le Poco F7 pendant au moins trente minutes : pauses, retours d'un intervenant, changements rapides, voix éloignée, bruit et chevauchements. Mesurer température et consommation. La réussite sur émulateur ne valide pas cette étape.
5. Tester renommage après plusieurs passages, homonymes, filtre/réaffichage, correction simultanée, suppression manuelle, nouvelle session et reprise. Aucune retouche perdue ; aucun prénom ne migre vers un autre profil lors d'une réinitialisation.
6. Tester une voix ignorée simultanée à une voix conservée. Conserver les mots attribuables à la personne retenue ; signaler l'incertitude lorsque la séparation n'est pas fiable.
7. Vérifier la dictée simple : aucun chargement du modèle de diarisation, comportement et performances inchangés à la variabilité de mesure près.

La première livraison technique doit être cette preuve de streaming réel. Une interface simulée ou un APK qui affiche seulement une liste d'intervenants ne constitue pas une validation de la fonction.

Sources techniques

- Nemotron 3 Diarization — fiche NVIDIA : huit locuteurs, mémoire de session, streaming et latence de tampon.
- Association à Nemotron 3.5 ASR : pipeline de transcription attribuée, avec limites pour les voix superposées.
- NeMo-Speech.cpp et SDK natif : interfaces C pour ASR et diarisation ; aucune performance Android déduite de leur seule disponibilité.

Mode Réunion en direct — plan d’implémentation

> Pour les agents d’implémentation : exécuter les tâches bornées selon la politique Sol–Luna de `/Users/ulliemaillot/CLAUDE.md`. Astra rédige la spec et ce plan, effectue la recherche, décide et réalise les revues. Luna 6 Max implémente et teste après attestation. Les instructions génériques des skills ne changent pas ce partage. Aucun commit ni push n’est autorisé par ce document.

Auteur : Astra, agent principal de la conversation. Rédaction non déléguée. 24 septembre 2026.

But : ajouter un mode Réunion qui transcrit pendant l’enregistrement, distingue jusqu’à huit voix, permet de renommer ou masquer un intervenant et protège les corrections manuelles.

Architecture : conserver la voie Dictée existante et ajouter une voie native Réunion. Le pont JNI fournit des mots horodatés et des attributions révisables à un document structuré ; les noms, retouches et filtres appartiennent au document, jamais à une chaîne de texte recomposée par remplacement de noms. L’overlay héberge un panneau de réunion dédié, piloté par un contrôleur séparé du moteur audio.

Stack : Kotlin Android natif, API minimale 30, arm64-v8a, JDK 21 pour Gradle, C++17/JNI, NDK 28.1.13356709, NeMo-Speech.cpp et GGML CPU privés, JUnit/Robolectric et instrumentation Android.

Spec : 2026-09-24-meeting-mode.md.

Espace d’exécution isolé : `/Users/ulliemaillot/.codex/worktrees/dictai-meeting-mode/phone-whisper`, créé depuis `26cb80d`. Les modifications d’images concurrentes du dépôt principal ont été intégrées exactement à la référence T0b. Elles ont depuis été enregistrées dans le commit principal `04268ac7a288fc32a95161cf7c6b418093be1db5` ; l’import initial des 35 fichiers a été vérifié par empreinte. Ce chantier Réunion reste isolé et ne remplace pas le checkout principal. Toute livraison commune exigera de comparer à nouveau les différences courantes.

Contraintes globales

- Mode choisi avant le démarrage, notamment dans le menu ouvert par un glissement vers le haut de la pastille. Dictée ne charge et n’exécute aucun modèle de diarisation.
- En Réunion : couronne de boucles cursives, rotation seulement pendant la capture ; appui pour pause/reprise ; glissement vers le bas depuis la pause pour enregistrer et terminer. Voir le contrat T7d.2 ci-dessous.
- Huit profils au maximum par réunion, y compris les profils ignorés. Une neuvième voix peut être confondue ; aucune garantie de détection supplémentaire.
- Les profils acoustiques vivent uniquement dans une session native. Une pause conserve cette session ; un arrêt du processus la perd.
- Les noms sont des métadonnées. Renommer actualise toutes les étiquettes et les exports sans toucher aux mots prononcés.
- Choix initial pour « Ignorer » : masquer réversiblement les interventions passées et futures. Les données internes restent conservées et peuvent être réaffichées.
- Texte non attribué visible sous « Intervenant à confirmer ». Une attribution incertaine ne permet pas de masquer arbitrairement des mots.
- Texte provisoire visible avant arrêt. Mesurer séparément délai des mots et délai des étiquettes ; ne pas promettre de délai nul ni une performance Poco issue de l’émulateur.
- Aucun effacement des retouches, suppressions, composition clavier, sélection ou images par une mise à jour automatique.
- Aucune inférence de locuteur par un modèle de nettoyage ; pas de chargement automatique du nettoyage local ou cloud en Réunion.
- Audio local, aucune bascule cloud. Pas de conservation permanente de l’audio par défaut. Files PCM bornées et erreurs visibles.
- Fichiers et modèles téléchargés vérifiés avant publication atomique. Aucun remplacement des bibliothèques natives de Dictée.
- Chaque lot : tests rouges, implémentation, tests verts, preuves. Selon la politique canonique, un lot moyen reçoit une revue Astra ; un lot complexe reçoit deux revues distinctes du même état final : conformité/sécurité, puis adversarial/maintenabilité/régressions.

Points de revue prioritaires

1. Une personne revient après un long silence : reprendre le même profil, jamais recycler un profil ignoré. Couvert par T2/T4/T8.
2. Une correction se déroule pendant une révision de mots ou de locuteur : préserver le texte et le curseur, y compris une suppression totale. Couvert par T3/T6.
3. Le processus meurt puis une note est rouverte : conserver les noms enregistrés sans les attribuer automatiquement à des voix nouvelles. Couvert par T5/T7.
4. Deux voix se chevauchent, dont une ignorée : montrer les mots non attribués, ne pas prétendre séparer les voix. Couvert par T2/T4/T8.
5. Le calcul prend du retard, le stockage manque ou l'utilisateur annule : pas de perte silencieuse ni de callback tardif dans une nouvelle réunion. Couvert par T1/T4/T5/T7.

Dépendances et ordre

`T0` → `T1` → `T4` → `T7` → `T8`. En parallèle de T1, `T2` → `T3` → `T5` → `T6` peut avancer sur des événements simulés clairement identifiés. T7 réunit les deux chemins uniquement après preuve native T1. Aucun écran simulé n'est une preuve de transcription réelle.

T1 est un critère de passage technique : si le runtime ne produit pas d'attributions avant la fin du flux sur Android, ne pas publier le mode Réunion comme fonctionnel. Astra documente la cause et choisit la correction technique ; les agents ne changent pas de modèle ou de périmètre de leur propre initiative.

Contrats communs

Nouveaux types dans `app/src/main/kotlin/com/kafkasl/phonewhisper/meeting/MeetingModels.kt` ; package `com.kafkasl.phonewhisper.meeting`.

```
data class MeetingWord(val text: String, val startMs: Long, val endMs: Long, val channel: Int)
data class MeetingHypothesis(
    val runId: String,
    val utteranceId: Long,
    val revision: Long,
    val words: List<MeetingWord>,
    val transcript: String,
    val isFinal: Boolean,
    val stableSpeakerThroughMs: Long,
    val audioProcessedMs: Long,
)
data class MeetingParticipant(val id: String, val ordinal: Int, val channel: Int,
    val name: String? = null, val ignored: Boolean = false)
data class MeetingTurn(val id: String, val utteranceId: Long, val startMs: Long,
    val endMs: Long, val recognizedText: String, val automaticParticipantId: String?,
    val manualParticipantId: String? = null, val hasManualAttribution: Boolean = false,
    val editedText: String? = null,
    val attributionStable: Boolean = false)
data class MeetingDocument(val schemaVersion: Int = 1, val sessionId: String,
    val runId: String, val participants: List<MeetingParticipant> = emptyList(),
    val turns: List<MeetingTurn> = emptyList(), val finished: Boolean = false)
```

`channel=0` signifie non attribué, `1..8` est le canal du modèle. Le numéro affiché est l'ordre de première découverte, pas nécessairement le canal. Les IDs de profils dérivent de la session et du canal ; ils ne traversent pas une nouvelle réunion. `editedText=null` signifie absence de retouche ; `editedText=""` signifie suppression volontaire, à conserver. Une attribution manuelle vers « à confirmer » utilise `hasManualAttribution=true` et `manualParticipantId=null`, sans confondre ce choix avec une absence de correction.

Le pont incrémente `utteranceId` après chaque résultat final, même vide, et `revision` à chaque publication. Les horodatages restent ceux de l'audio capturé, sans ajouter la durée des pauses. Il conserve uniquement les résultats finaux dont les étiquettes peuvent encore être révisées, les actualise avec `refresh_speaker_tags` et les libère lorsqu'ils passent la frontière `stable_speaker_time`. Les événements d'une ancienne `runId` ou d'une révision déjà appliquée sont rejetés.

T0 — Établir la référence de travail

Responsabilité : Luna pour les commandes de build et tests ; Astra pour le plan et l'état d'avancement. Difficulté moyenne.

Fichiers : aucun changement fonctionnel ; preuves dans `docs/research/meeting-mode/validation.md`, sorties brutes sous `app/build/reports/meeting/`.

- [x] Consigner HEAD, état Git, Java, ABI ADB, versions NDK et Gradle. Configurer `local.properties` dans le worktree avec `sdk.dir=/Users/ulliemaillot/Library/Android/sdk`.
- [x] Exécuter la référence avec `JAVA_HOME='/Applications/Android Studio.app/Contents/jbr/Contents/Home' ./gradlew :app:testDebugUnitTest :app:assembleDebug`.
- [x] Conserver les échecs préexistants avec leur commande exacte. Ne pas modifier les fonctions images pour rendre ce chantier vert.
- [x] Vérifier que les bibliothèques existantes sont disponibles ; consigner leurs SHA-256 pour contrôler qu'elles ne changent pas pendant T1.

T1 — Prouver le moteur Réunion sur Android arm64

Responsabilité : Luna native, bornée au build, à la sonde et à ses preuves. Difficulté complexe.

Créer : `scripts/build_meeting_android.sh`, `scripts/native/meeting-android.patch`, `app/src/main/cpp/meeting/meeting_probe.cpp`, `app/src/main/cpp/meeting/CMakeLists.txt`, `scripts/check_meeting_native.sh`. Staging ignoré : `.native-cache/meeting/`. Ne pas toucher au moteur JNI actuel.

Entrées figées :

Composant	Révision / fichier	Taille et SHA-256
Runtime	`NVIDIA/NeMo-Speech.cpp` à `97a15afa5caa9bce5baaa86c1184103877af4101` ; GGML au gitlink de cette révision	Enregistrer le gitlink dans le rapport
SentencePiece statique	`google/sentencepiece` à `17d7580d6407802f85855d2cc9190634e2c95624`	Sources épinglées, compilation Android
ASR	`nvidia/nemotron-3.5-asr-streaming-0.6b`, révision `1c8deaecc64b91f034d73e08dd8b64625eb3395d`, `nemotron-3.5-asr-streaming-0.6b.q8_0.gguf`	741548352 ; `a5c435f294eea8f88ce68dd27b8c3bfea7f777cb2fbba04fcd30eaa555f429ae`
Diarisation	`nvidia/Nemotron-3-Diarization`, révision `f667ed73aee57d40cc39428eb768b4fd87a0a29e`, `Nemotron-3-Diarization.q8_0.gguf`	107012128 ; `08456d9e22cd9a323c0364d98375f3746d6e68507ebb705cd46438c534c7a3a1`

Sources primaires vérifiées par Astra : `models/index.json`, `src/asr/recognizer.{h,cpp}`, `src/asr/c_api.cpp`, `app/transcribe.cpp`, `scripts/build_sentencepiece_static.sh` de la révision figée. Copie de recherche disponible dans `/tmp/dictai-meeting-source.Wm4OhB` ; le script final doit pouvoir reconstruire sans ce chemin temporaire.

Les fiches ASR à la révision retenue et diarisation à la révision retenue déclarent toutes deux OpenMDW-1.1. Inclure leur provenance et l'accord dans les notices du paquet Réunion ; les licences du code NeMo-Speech.cpp, GGML et SentencePiece restent distinctes. Les poids ne sont pas incorporés à l'APK.

- [x] Écrire le contrôle rouge : l'exécutable de sonde manque, ou ne produit aucun événement `speaker_tag>0` avant `finish`. Le test doit distinguer cette situation d'un résultat final correct.
- [x] Compiler uniquement ASR/diarisation CPU, C++17, API 30, `arm64-v8a`, STL statique, pages ELF 16 Ko. Désactiver TTS/NMT/S2S/HTTP/gRPC/micro CLI/Metal/CUDA/Vulkan/BLAS/OpenMP non disponible et `GGML_NATIVE`. Utiliser `NEMO_SPEECH_GGML_PATCHED=OFF` pour ce chemin CPU.
- [x] Absorber GGML et SentencePiece statiquement dans une bibliothèque privée. Le patch de build peut rendre les cibles nécessaires statiques ; ne pas modifier les algorithmes upstream. Masquer les symboles internes via visibilité et script de version. Le résultat final JNI sera `libdictai_meeting.so`, sans dépendance sur les `libggml*.so` de Dictée ni SONAME `.so.1` non empaquetable.
- [x] La sonde utilise l'API C++ `Recognizer::streaming_recognize`, `set_interim_words(true)`, `push`, `next`, `finish`, `stable_speaker_time` et `refresh_speaker_tags`. L'API C de la révision épinglée n'active pas les mots intermédiaires : ne pas supposer le contraire.
- [x] Vérifier la capacité réelle via `recognizer.diar_model()` et les métadonnées de capacité avant d'accepter le modèle. Rejeter un modèle à quatre voix même si une option demande huit.

- [x] Alimenter des blocs PCM mono 16 kHz de 160 ms ; cadence temps réel avec horloge monotone, puis essai sans cadence pour mesurer le débit. Séparer la durée de chargement et d'échauffement de l'inférence.
- [x] Premier témoin reproductible : audio public upstream `test_files/diar/ami_en2002d_2132.wav`, empreinte enregistrée. Il vérifie le mécanisme en anglais. Ajouter ensuite un témoin français à trois voix ; des voix de synthèse sont acceptables pour la plomberie, explicitement insuffisantes pour la qualité de diarisation humaine.
- [x] Comparer ASR seul et ASR+diarisation sur le même audio. Enregistrer chaque résultat avec temps monotone, audio fourni/traité, final/provisoire, timestamps et tags ; sans enregistrement privé de l'utilisateur.
- [x] Contrôler `llvm-readelf` : ABI, dépendances, alignement ; `llvm-nm` : absence d'exports GGML. Charger ancien et nouveau runtime dans le même processus Android avant de conclure à leur coexistence.
- [x] Publier la preuve : tags non nuls avant EOF, au moins deux tags sur le témoin multi-voix, changement puis retour d'un locuteur ; nombre de voix effectif du modèle égal à huit. Rapporter séparément toute erreur d'attribution. Ne pas conclure « direct utilisable sur Poco » à partir de ce test.

Sortie requise : sources de la sonde et du build reproductibles, binaires arm64 vérifiés, journal avec événements intermédiaires, décision technique Astra autorisant T4/T7 ou correction bornée de T1.

T2 — Profils de session et projection des interventions

Créer : `meeting/MeetingModels.kt`, `meeting/MeetingParticipants.kt`, `meeting/MeetingProjection.kt` ; tests homonymes dans `app/src/test/kotlin/com/kafkasl/phonewhisper/meeting/`.

API : `MeetingParticipants(sessionId: String)`, `observe(channel: Int): MeetingParticipant?`, `rename(id: String, name: String)`, `setIgnored(id: String, ignored: Boolean)`, `all(): List<MeetingParticipant>` ; `MeetingProjection.text(document: MeetingDocument): String`.

- [x] Écrire et exécuter le test rouge suivant, puis les cas canal 0/canal 9, huit canaux, homonymes, nouveau sessionId, profil ignoré qui revient :

```
@Test fun ignoredParticipantKeepsItsIdentity() {
    val profiles = MeetingParticipants("meeting-a")
    val first = requireNotNull(profiles.observe(6))
    profiles.rename(first.id, "Sophie")
    profiles.setIgnored(first.id, true)
    val returned = requireNotNull(profiles.observe(6))
    assertEquals(first.id, returned.id)
    assertEquals("Sophie", returned.name)
    assertTrue(returned.ignored)
    assertEquals(1, profiles.all().size)
}
```

- [x] Implémenter IDs internes stables, ordre de découverte, nom vide → libellé par défaut, nom limité à 80 caractères, profil conservé lors du filtrage. Aucun profil pour un canal hors de l'intervalle 1..8.
- [x] Projection : étiquette sur sa propre ligne puis texte visible ; regrouper seulement des prises de parole adjacentes du même profil si cela ne supprime pas les IDs d'édition. Exclure les interventions d'un profil ignoré uniquement quand son attribution est stable ou manuelle. Conserver les autres sous « Intervenant à confirmer ».
- [x] Tester que renommer Sophie ne remplace pas « Sophie » dans les phrases, qu'un homonyme conserve un ID distinct et que `editedText=""` reste vide après projection. Omettre les tours au corps vide ou blanc du texte exportable, sans supprimer leurs ancres dans le document ; ne pas exporter une étiquette orpheline après suppression manuelle. Tester aussi le réaffichage exact d'un texte retouché, l'attribution manuelle à un profil conservé face à un profil automatique ignoré, et un prénom contenant des caractères Unicode supplémentaires à la limite des 80 points de code.
- [x] Exécuter `./gradlew :app:testDebugUnitTest --tests '*meeting.MeetingParticipantsTest' --tests '*meeting.MeetingProjectionTest'` ; fournir preuve rouge/verte.

T3 — Réconcilier les hypothèses sans écraser l'édition

Créer : `meeting/MeetingTranscriptReducer.kt`, `meeting/MeetingEditAnchor.kt` et leurs tests. Réutiliser l'algorithme d'alignement d'`EditableTranscript.kt` lorsque pertinent, sans changer le comportement Dictée.

API : `MeetingTranscriptReducer(sessionId: String, runId: String)`, `apply(hypothesis: MeetingHypothesis)`, `edit(turnId: String, text: String)`, `assign(turnId: String, participantId: String?)`, `rename(participantId: String, name: String)`, `setIgnored(participantId: String, ignored: Boolean)`, `snapshot(): MeetingDocument`.

- [x] Utiliser le champ `hasManualAttribution` défini dans T2 pour conserver une attribution manuelle vers « à confirmer » au callback suivant.

- [x] Test rouge : hypothèse « mardi », édition « jeudi », révision ASR « mardi matin », renommage, filtrage/réaffichage ; le rendu conserve « jeudi » et les paroles nouvelles, sans dupliquer l'énoncé.
- [x] Test rouge : suppression totale d'une prise de parole, puis nouvelle hypothèse/final ; le texte supprimé ne ressuscite pas. Une hypothèse d'une ancienne `runId` ou révision ne change rien.
- [x] Identifier une prise de parole par `utteranceId` et ancre audio stable. Réconcilier d'abord les mots par ordre et recouvrement temporel, puis construire les frontières de voix. Ne pas utiliser le prénom ou l'index de ligne comme clé. Une prise de parole retouchée garde son ID et son ancre ; ses changements automatiques de segmentation ne doivent pas scinder ni déplacer la retouche.
- [x] Les IDs d'utterance sont strictement croissants dans une session native ; cet ordre prévaut lorsque les timestamps d'une hypothèse sont absents. La naissance d'un tour reçoit un identifiant unique de session, même si deux tours ont les mêmes timestamps. Tester la transition d'une hypothèse sans mots vers des mots qui dépassent l'ancienne frontière audio, sans duplication de la continuation.
- [x] Conserver les retouches comme surcharges du texte reconnu, avec ancre de fin au dernier mot couvert ; ajouter les mots nouveaux après cette ancre. Une révision sous l'ancre ne réécrit pas la surcharge. Borner l'alignement aux hypothèses actives, jamais à toute la réunion.
- [x] Réviser seulement les métadonnées de voix des passages déjà édités. Une correction manuelle de voix a priorité sur le modèle. Une révision d'une frontière de diarisation peut laisser un passage composite « à confirmer » ; elle ne peut pas attribuer arbitrairement le texte retouché à une voix ignorée.
- [x] Tests : final reçu deux fois, fin vide, ponctuation tardive, zéro mot mais transcript non vide, mot sans timestamps utilisables, retour au même locuteur, changement de canal provisoire, suppression d'un mot ancien et ajout d'une phrase nouvelle.
- [x] Exécuter `./gradlew :app:testDebugUnitTest --tests '*meeting.MeetingTranscriptReducerTest' --tests '*meeting.MeetingEditAnchorTest'` ; aucune perte de texte non attribué.

T4 — Pont JNI, chargement et session audio bornée

Créer : `app/src/main/cpp/meeting/meeting_jni.cpp`, `meeting/MeetingNative.kt`, `meeting/MeetingEngine.kt`, `meeting/MeetingAudioQueue.kt`, `meeting/MeetingModelCatalog.kt`, `meeting/MeetingModelStore.kt` et tests. **Modifier :** configuration CMake/Gradle pour empaqueter uniquement le nouveau binaire et licences nécessaires.

API Kotlin : `MeetingEngine` reçoit les deux chemins de modèles validés et un `MeetingNativeBridge` injectable. `start(runId, language, onReady, onUpdate, onFailure): MeetingSession` lance un worker unique propriétaire du handle natif. `MeetingSession` expose `acceptPcm16(buffer, length): Boolean`, `finish()`, `cancel()`, `close()` et une complétion asynchrone de fermeture. Le micro attend `onReady`, après chargement et échauffement. `acceptPcm16` retourne false avant cet état, après fermeture ou saturation, sans bloquer le lecteur micro.

Cycle de session : `finish()` termine l'entrée, draine les blocs, publie les derniers résultats puis ferme le handle. `cancel()` et `close()` invalident les prochains callbacks et vident la file ; si le natif charge ou calcule déjà, le worker le laisse sortir puis ferme lui-même le handle. Aucun autre thread ne détruit un objet natif occupé. La complétion de fermeture est résolue après la libération effective, même en échec, et le contrôleur l'attend avant de recharger Dictée. L'admission d'un callback est vérifiée sous verrou, puis son exécution a lieu hors verrou sur le worker ; un callback déjà admis peut terminer sans bloquer `cancel()` ou l'offre PCM. Le contrôleur vérifie encore `runId` et son état après toute remise sur le thread principal. Un arrêt demandé avant `onReady` n'allume jamais le micro. Une pause arrête seulement la capture ; le worker draine puis attend sans finaliser le stream. Erreurs uniques et lisibles, y compris `UnsatisfiedLinkError`, sans paroles ni chemins sensibles dans les messages. La saturation provoque un arrêt visible de la capture par le contrôleur, puis la finalisation des blocs déjà acceptés ; aucune boucle ne supprime discrètement des blocs pour rattraper le temps.

Une instance de moteur n'admet qu'une session à la fois. La complétion de fermeture est exceptionnelle si le pont ne peut pas confirmer la destruction du handle ; dans ce cas le contrôleur ne recharge aucun autre gros modèle et garde un état d'erreur explicite. Un échec d'ouverture ou de traitement suivi d'une fermeture réussie libère normalement la place. Les tests utilisent un faux pont bloquant pour vérifier annulation pendant chargement/calcul, absence de destruction concurrente, ordre FIFO, saturation, finalisation unique et suppression des retours tardifs.

Frontière native précisée après T1 : `MeetingNativeBridge` expose `open(asrPath, diarPath, language)`, `acceptPcm16(handle, buffer, length)`, `finish(handle)` et `close(handle)`. Les résultats contiennent les champs de `MeetingHypothesis` sauf `runId`, ajouté par le worker Kotlin propriétaire. Aucun callback C++ ne touche l'UI. La façade `JniMeetingNative` charge `libdictai_meeting.so` uniquement à la demande. Le transport textuel JNI utilise des octets UTF-8 ou des objets correctement convertis ; éviter `NewStringUTF` pour du texte Unicode arbitraire. Les handles sont validés dans un registre et ne sont jamais des pointeurs directement déréférencés depuis un entier Kotlin.

Le pont conserve la dernière finale stable assez longtemps pour rattacher une éventuelle `late_punctuation` à son ancienne utterance par une nouvelle révision. Il borne les finales provisoires et impose un endpoint au plus tard après environ trente secondes d'audio dans une utterance, sans recréer la mémoire des locuteurs. Le chargement et l'échauffement terminent avant que le contrôleur n'ouvre le micro ; leur annulation invalide immédiatement les callbacks, puis libère les ressources dès le retour de l'opération native en cours.

- [x] Tests rouges avec un faux pont : file bornée à 10 secondes de PCM (320 000 octets à 16 kHz/16 bits), copies des buffers réutilisés par AudioRecord, dépassement déclaré, annulation pendant chargement, fermeture idempotente, aucun callback après invalidation de `runId`.
- [x] JNI : ownership RAII, validation des tailles et handles, conversion PCM16LE, copie des mots avant libération native, exceptions C++ transformées en erreurs Kotlin. Aucun pointeur libéré pendant un décodage ; annulation coopérative et destruction uniquement sur le worker.
- [x] Fenêtres ASR finales comptées explicitement ; préserver les horodatages absolus. Réviser les attributions des finales en attente avec `refresh_speaker_tags`, publier la frontière stable et retirer les buffers devenus stables. Configurer un endpoint avant que la taille d'hypothèse ne croisse avec toute la réunion, en conservant le même objet de diarisation.
- [x] Catalogue : URL Hugging Face `/resolve/<revision>/<filename>` pour les deux modèles de T1. Téléchargement annulable vers `.part`, taille+SHA, publication atomique dans un dossier Réunion distinct. Espace disque insuffisant, modèle tronqué, empreinte erronée et interruption ne produisent jamais l'état « Prêt ».
- [x] Tests de stockage : ancien paquet complet préservé après échec du nouveau, annulation nettoyant seulement son dossier temporaire, fichiers attendus exigés tous les deux. L'UI affiche environ 849 Mo de téléchargement ; poids non incorporés à l'APK.
- [ ] Chargement à la demande. Libérer le modèle Dictée avant le chargement Réunion pour éviter deux Nemotron résidents ; restaurer le chargement normal de Dictée à la sortie. Ne pas charger de formateur LLM.
- [ ] Si seule l'attribution devient indisponible, conserver le texte et marquer les nouvelles attributions inconnues. Si l'intégration native ne permet pas d'isoler la panne, arrêter visiblement cette session et préserver le brouillon. Une reprise explicite crée une nouvelle session ; ne pas réutiliser les canaux perdus ni présenter cette reprise comme une continuité acoustique.
- [ ] Validation : tests de file/session/stockage, instrumentation de chargement simultané des deux runtimes, modèle à huit voix vérifié, absence de chargement Réunion en Dictée.

T4d — Gestion commune des deux modèles

`MeetingModelCatalog` fige le nom du paquet, les deux URLs versionnées, tailles, empreintes et chemins relatifs sûrs. `MeetingModelStore` fournit une instance partagée par stockage privé de l'application, réutilisée par l'assistant et l'accueil. Son état observable distingue absent, vérification, téléchargement, prêt et échec ; les abonnements UI sont retirés à la destruction de leur propriétaire. L'inspection complète et le téléchargement se déroulent sur un worker dédié. Les commandes idempotentes `refresh`, `download`, `cancelDownload` ne déclenchent jamais le micro et n'exposent aucun chemin privé dans les erreurs affichées.

Le dossier final est identifié par le paquet versionné. Une inspection vérifie taille et SHA-256 des deux fichiers ; aucun simple test d'existence n'accorde l'état prêt. Le téléchargement écrit dans un dossier temporaire propre à l'opération et vérifie chaque flux, puis publie le dossier complet par renommage sur le même système de fichiers. Un ancien paquet valide reste utilisable si un nouveau téléchargement échoue. Vérifier l'espace libre pour le paquet temporaire et une marge raisonnable avant de commencer, puis propager proprement un manque d'espace en cours de transfert. L'annulation ferme l'appel OkHttp actif, invalide les callbacks de cette opération et nettoie seulement son propre temporaire. Une réponse trop grande est interrompue dès le dépassement de la taille attendue. Les modèles restent téléchargeables sans clé ; ne jamais réutiliser les credentials des autres fournisseurs de l'application.

Les tests injectent des petits artefacts et un serveur HTTP local : succès des deux fichiers, second fichier absent/tronqué, hash incorrect, annulation d'un flux bloqué, manque de place, publication interrompue, conservation du paquet précédent et commandes concurrentes sans second téléchargement. L'intégration UI utilise les chemins d'un état prêt obtenu hors du thread principal.

T4f — Point de sauvegarde après vidage de la file

Compléter `MeetingSession` avec `checkpoint(): CompletableFuture<Unit>` pour la pause et l'export pendant une session. Cette barrière capture le nombre de blocs acceptés à l'instant de l'appel et se résout lorsque le worker les a tous soumis au natif et a transmis leurs éventuelles mises à jour. Elle ne finalise pas le stream et ne recrée aucun profil. Un bloc accepté plus tard appartient au point de sauvegarde suivant. Le comptage de l'admission et de la consommation est atomique ; ne pas déduire ce point de la seule taille de file, puisqu'un bloc peut être en calcul.

La pause arrête d'abord le lecteur AudioRecord, demande ce point, puis revient sur le thread principal après les mises à jour déjà postées et vide le brouillon. Le statut peut indiquer « Mise en pause… » pendant cette attente ; les retouches restent possibles. La barrière ne prétend pas produire des mots que le modèle garde encore dans sa fenêtre interne. Une annulation ou un échec invalide les barrières en attente avec une erreur constante et aucune attente perpétuelle ; une fermeture normale les termine une fois les blocs effectivement traités. Les complétions sont exécutées hors verrou. Tests : bloc natif en cours malgré une file vide, appels simultanés, audio ajouté après la demande, aucune donnée, fin normale, annulation et échec natif.

T5 — Sauvegarde structurée et exports compatibles

Créer : `meeting/MeetingDocumentJson.kt`, `meeting/MeetingDraftStore.kt` et tests. **Modifier :** `TranscriptNotes.kt`, `AndroidTranscriptNoteStorage.kt`, `NoteShareText.kt`, `NoteHtmlExport.kt`, `NotePdfExport.kt` uniquement aux frontières de lecture de la projection.

Contrat : `TranscriptNote` reçoit un champ optionnel final `meeting: MeetingDocument? = null`. Une méthode explicite `saveMeeting(id: String?, meeting: MeetingDocument, images: List<NoteImage>? = null): TranscriptNote` conserve dossiers/images/titre comme `save`. `note.text` contient toujours la projection visible la plus récente pour les consommateurs historiques. Les champs structurés restent la source de vérité.

Conserver également un payload opaque pour les versions inconnues afin qu'un renommage ou un changement de dossier ne les efface pas. Une note structurée se rouvre dans le panneau Réunion ; le chemin d'édition plate ne peut pas modifier silencieusement son texte sans sa structure.

Identité de stockage : une nouvelle note Réunion prend `meeting.sessionId` comme identifiant ; un identifiant déjà occupé par une autre note est refusé. Les mises à jour doivent viser la même session. Le titre automatique provient du premier corps visible contenant du texte, sans étiquette de locuteur ; une réunion vide reçoit « Nouvelle réunion ». Une sauvegarde plate d'une note structurée ou opaque est refusée explicitement. Les opérations de titre/dossier conservent tous les champs meeting. La publication d'une note Réunion utilise un enregistrement vérifié et propage les échecs ; l'association des images ne peut pas être réputée durable sur la seule base d'un `SharedPreferences.apply()`.

Format et consommateurs : le champ JSON `meeting` contient une chaîne portant le document sérialisé. Cela permet de conserver exactement un payload inconnu ou invalide dans `meetingRaw`, sans le reformater. Le document reconnu et le payload opaque sont mutuellement exclusifs. La lecture de version 1 et l'entrée des exports recalculent la projection depuis la structure et les pièces jointes ; un ancien `note.text` ne doit pas faire réapparaître les paroles d'une voix ignorée.

Images : réutiliser les métadonnées `NoteImage` et la sérialisation des blocs déjà présents dans le projet. Les corps édités peuvent contenir leurs marqueurs internes, rendus comme objets graphiques par l'éditeur. La projection d'une voix ignorée retire ses paroles et son étiquette, mais émet encore les blocs d'images connus à cette même place dans l'ordre du document. Les images sont ainsi indépendantes du filtre vocal. Passer explicitement les numéros des images réellement jointes à cette projection ; un marqueur saisi sans pièce jointe ne crée pas d'image. La livraison d'une capture enregistre les pièces jointes avec la note de cette session avant de détacher sa réservation persistante. Le brouillon atomique conserve les derniers tours ; à la récupération, il retrouve les pièces jointes par le même identifiant de réunion. Tester interruption entre ces opérations, filtre/réaffichage et ordre d'export.

- [x] Tests rouges : aller-retour JSON de noms homonymes, ignorés, retouches vides, attribution manuelle inconnue et session terminée ; une ancienne note sans champ meeting se charge normalement.
- [x] Sérialiser la version 1, profils, tours, retouches et état ; ne jamais sérialiser des handles natifs ou un prétendu profil biométrique. Une version future inconnue conserve le texte de repli et reste lisible, sans écraser silencieusement la structure inconnue.
- [ ] Sauvegarder le brouillon Réunion atomiquement dans un fichier privé distinct, écriture regroupée à 500 ms maximum et vidage avant pause/arrêt/export. Éviter de réécrire toutes les préférences sur chaque mot. Échec disque : conserver le document en mémoire et afficher « Sauvegarde impossible ».
- [x] Vérifier la publication effective du fichier : `AtomicFile.finishWrite` ne propage pas tous les échecs. Un fsync explicite et un contrôle de publication doivent empêcher un faux succès. Récupérer aussi la première écriture interrompue qui n'a laissé qu'un fichier temporaire ; conserver tout payload inconnu ou invalide jusqu'à une action explicite de l'utilisateur.
- [ ] Rouvrir une réunion enregistrée en lecture/édition avec ses noms. Reprendre le micro après destruction du processus crée une nouvelle réunion liée ou exige une nouvelle session affichée ; ne jamais mapper les canaux nouveaux sur les anciens noms automatiquement.
- [ ] Exports texte/HTML/PDF utilisent la projection courante et omettent les paroles ignorées ; les prénoms Unicode sont conservés et échappés en HTML. Les images restent dans leur ordre et ne sont pas supprimées par le filtre d'un locuteur.
- [ ] Exécuter les tests notes existants plus `'*meeting.MeetingDocumentJsonTest'`, `'*meeting.MeetingDraftStoreTest'`, tests exports. Vérifier sauvegarde, changement de dossier et suppression de dossier sur une note Réunion.

T5d — Édition d'une note rouverte et regroupement des écritures

Créer : `meeting/MeetingDocumentEditor.kt`, `meeting/MeetingDraftWriter.kt` et leurs tests ; extraire au besoin la normalisation des prénoms de `MeetingParticipants` vers une fonction commune, sans changer ses règles.

`MeetingDocumentEditor` offre une seule frontière `snapshot`, `apply`, `edit`, `assign`, `rename`, `setIgnored`, `finish` au contrôleur. Une création neuve encapsule le reducer vivant avec ses propres sessionId/runId ; une restauration reçoit un document validé, conserve toutes ses identités et autorise uniquement les retouches de document. Aucun événement ASR ne peut être appliqué à un document restauré, même si son runId ressemble à celui du callback. L'arrêt final passe le document à `finished=true` après les dernières hypothèses. Les modifications de noms et de corps restent possibles ensuite. Une attribution manuelle ne peut référencer qu'un participant existant ou l'inconnu explicite. La consultation du brut reste disponible indépendamment des filtres. La méthode idempotente `ensureDocumentTurn(): MeetingTurn` crée au besoin un tour documentaire vide et non attribué, avec un ID stable distinct et une utterance réservée à zéro ; les utterances natives commencent à un. Elle existe aussi en édition restaurée et peut servir après les premières paroles si aucune ancre éditable n'est sélectionnée. Ce tour reste avant les premières paroles et n'est jamais remplacé par l'ASR. Dès qu'un tour existe, l'ancrage de capture utilise son ID et son curseur sérialisé ; une image ne nécessite pas de créer un faux profil vocal.

`MeetingDraftWriter` possède un worker d'E/S unique et une seule référence au dernier snapshot en attente. Le délai de 500 ms part de la première mutation non enregistrée : chaque nouveau mot remplace le snapshot en attente sans repousser indéfiniment l'écriture. `flush()` renvoie une complétion asynchrone couvrant au moins la génération courante ; pause, fin et export attendent cette complétion sans bloquer l'UI. Des mutations nouvelles pendant une écriture sont regroupées pour le passage suivant, sans file de copies du document entier. Une erreur est rendue au contrôleur, conserve le snapshot en mémoire et rend le flush exceptionnel ; une nouvelle modification ou une nouvelle demande explicite de flush peut réessayer. La fermeture refuse les nouvelles mutations et sérialise le dernier flush avant la libération du worker. Si ce flush échoue, sa complétion est exceptionnelle, le dernier snapshot reste disponible et une nouvelle demande explicite peut réessayer ; aucune boucle automatique ni fermeture réputée réussie. Un callback ancien est ignoré par le contrôleur après fermeture ou changement de session.

Le constructeur, y compris sa variante publique recevant le magasin de brouillon, accepte `onErrorChanged: (String?) -> Unit` avec un callback vide par défaut. Un échec d'écriture automatique notifie un message constant « Sauvegarde impossible », sans détail du stockage ; une écriture ultérieure réussie notifie `null` pour enlever cet état. Ces notifications s'exécutent hors verrou et une exception du callback ne perturbe ni l'écriture ni les complétions. Le contrôleur les republie sur le thread principal après vérification de l'identité de session. Une future de `flush()` ne suffit pas à signaler une panne survenue pendant les sauvegardes automatiques.

Tests : édition hors ligne sans acceptation ASR ; retouches vides/noms homonymes/filtre ; fin rejetant les retours tardifs ; débit continu qui n'affame pas la sauvegarde ; dernière génération effectivement écrite avant résolution de flush ; écriture bloquée avec remplacement du snapshot en attente ; panne persistante sans boucle de tentatives ; notification d'une panne automatique et effacement après reprise, sans fuite du message interne et malgré un callback défaillant ; reprise après échec et fermeture sans perte de la dernière modification. Utiliser un stockage et un ordonnanceur injectables, avec attentes bornées.

T6 — Panneau Réunion éditable pendant l'enregistrement

Créer : `meeting/MeetingPanelView.kt`, `meeting/MeetingTurnEditor.kt`, `meeting/MeetingPanelController.kt` et tests Robolectric/instrumentation. **Modifier :** ressources de chaînes si nécessaires. Ne pas ajouter toute la logique dans `OverlayService.kt`.

Contrat : le contrôleur reçoit un document immuable, des callbacks `edit`, `assign`, `rename`, `setIgnored` et une action de téléchargement. Les lignes utilisent des IDs stables ; chaque étiquette est séparée de son corps éditable. Le panneau réutilise les couleurs, tailles et conventions du carnet actuel. Sa palette provient de `ThemeTokens.palette(context)` pour respecter le choix clair/sombre manuel aussi bien que le système ; `refreshTheme()` actualise les vues existantes sans recréer le champ focalisé, et l'hôte l'appelle lors du changement de thème. Une révision ambiguë après une retouche conserve chaque corps visible une seule fois et donne accès au texte reconnu par une action explicite ; elle ne réinsère pas l'énoncé complet dans un autre champ.

`MeetingPanelStatus.saveError: String?` porte une erreur de sauvegarde indépendamment de la phase audio ; `MeetingPanelActions.retrySave` déclenche la tentative explicite. Afficher cette erreur et « Réessayer », dans le menu si la hauteur est réduite. Une panne disque pendant l'écoute conserve les commandes de cette écoute : elle ne transforme pas la phase en erreur audio et ne propose pas un second démarrage. La réussite d'une écriture remet `saveError` à null.

Projection commune : exposer des lignes contenant `turnId`, étiquette optionnelle, corps, état d'attribution et possibilité d'éditer les paroles. `text()` exporte ces mêmes lignes. Les corps vides peuvent être conservés pour l'édition active sans exporter une étiquette vide. Une ligne filtrée contenant des images connues expose seulement ces blocs, sans étiquette

vocale et sans champ permettant d'éditer les paroles masquées. Les actions d'images restent disponibles via des commandes dédiées lorsque la ligne ne permet pas l'édition des paroles ; désactiver réellement la saisie, y compris au clavier matériel. Pour rendre un tour avec `TranscriptImageBlocks.fromNote`, ne passer que les pièces jointes référencées dans ce tour : cette méthode ajoute les images non référencées en fin de texte. Une image connue est rendue une seule fois dans le document ; les pièces jointes sans ancre sont présentées une seule fois en fin de réunion. Le panneau reçoit les pièces jointes et réutilise `OverlayTranscriptEditor` pour les objets graphiques ; un marqueur inconnu reste du texte ordinaire. Une seule logique décide de l'attribution, du filtrage et de l'ordre, partagée par UI et exports.

Liste et dialogues : utiliser RecyclerView 1.3.2, compatible avec compileSdk 34, avec IDs stables et animations de remplacement désactivées pour les champs focalisés. Le propriétaire de l'overlay fournit la présentation des dialogues et leur type de fenêtre. Les interactions ne dépendent pas d'une Activity implicite. Les retours d'édition portent l'ID du tour ; les ajouts d'images capturent cet ID et l'ancre du curseur avant l'ouverture de la caméra.

Chaque choix de dialogue possède un ID String non nul. Le retour `null` signifie uniquement abandon : l'option « Inconnu » utilise un ID sentinelle explicite, converti ensuite en attribution manuelle nulle. Ignorer un résultat absent de la liste proposée. Capturer sessionId/runId à l'ouverture et ignorer les retours après changement de document ou destruction du panneau. Avant une attribution tardive, vérifier aussi que le tour existe encore, car une segmentation ASR peut remplacer un tour non retouché pendant l'ouverture du dialogue. Tester annulation sans mutation, choix explicite Inconnu, changement de session et disparition du tour cible.

Hauteur réellement disponible : la fenêtre minimale existante mesure 240 × 160 dp et sa barre de déplacement, agrandissement et masquage occupe 48 dp. Tester aussi le panneau Réunion dans son espace intérieur de 240 × 112 dp. À cette taille, réunir les commandes dans une seule rangée de 48 dp et placer les actions secondaires dans un menu ; conserver le statut accessible et au moins deux lignes de texte lisibles avec la taille de police nominale. Avec une police système agrandie, garder sa taille et permettre le défilement ainsi que l'agrandissement de la fenêtre ; ne pas réduire automatiquement la police pour tenir dans la hauteur minimale. Ne pas empiler un en-tête, un statut et une seconde rangée d'actions au détriment de l'éditeur. Dans les tours compacts, placer le bouton d'intervenant à côté du corps : une étiquette de 48 dp au-dessus cacherait ces deux lignes. Ce bouton garde une cible d'au moins 48 dp et une description accessible complète, même si son libellé visible est abrégé. Le passage compact/normal change l'agencement sans recréer le champ focalisé. L'hôte masque ses commandes propres à Dictée et n'ajoute pas une seconde barre de commandes Réunion. Les gouttières réservées au redimensionnement ne recouvrent pas les champs éditables.

Ambiguïtés d'édition : une attribution manuelle protège aussi la portée de son passage, même sans correction de mots. Si une révision sans timestamps ne s'aligne plus, conserver les corps existants et le texte brut consultable ; ne pas élargir l'attribution manuelle à tout l'énoncé. Le panneau propose « Passages à vérifier » au niveau de la réunion, accessible même si le tour portant ce texte brut appartient à une voix ignorée. Les paroles incertaines ne sont pas attribuées automatiquement à cette voix. Cette vue distingue la transcription proposée du texte corrigé conservé.

Cohérence avec le clavier : sérialiser les mutations du document sur le thread principal. Avant d'appliquer une hypothèse native, recueillir l'édition du champ focalisé contre l'état qu'il affichait, puis appliquer l'hypothèse et la différence de rendu. Éviter un diff asynchrone qui laisserait le champ afficher une ancienne révision pendant que son callback d'édition protège déjà une révision plus récente. Pour une continuation ajoutée après une zone en composition, conserver explicitement les bornes et objets des spans de composition et la sélection ; l'IME ne doit pas absorber les nouvelles paroles dans son mot en cours. Les IDs RecyclerView sont des identifiants longs uniques attribués aux IDs de tours, jamais leur seul `hashCode()`. Tester ce flux complet, pas seulement un renommage de TextView.

- [ ] Test rouge : enregistrement simulé actif, focus au milieu d'un ancien passage, nouveau mot et renommage ; l'ancien champ garde texte, sélection, composition IME et focus. Aucun `setText` global sur le document.
- [ ] Afficher le choix Dictée/Réunion avant départ, une liste compacte « Personne 1 », « Personne 2 », etc. ; actions « Renommer », « Ignorer », « Réafficher ». Montrer le libellé « Masquer ses interventions dans cette réunion » pour la portée du filtre.
- [ ] Toucher une étiquette ouvre l'attribution manuelle ; profils homonymes distingués par leur numéro. Toucher un profil ouvre ses actions. Les commandes restent disponibles micro actif et ne changent pas sa session.
- [ ] Mettre à jour uniquement les lignes concernées. Si une ligne en cours de composition devrait être masquée, achever/conserver la composition avant retrait ; pas de perte de caractères. Le défilement suit le bas seulement si l'utilisateur y était déjà.
- [ ] Mode téléchargement : taille, progression et annulation ; mode chargement : état visible, annulation possible ; attribution provisoire visible ; modèle indisponible et retard audio annoncés sans bloquer les contrôles.
- [ ] Recycler ou paginer les prises de parole longues ; éviter un EditText conservé pour chaque tour d'une réunion entière. Préserver l'ancre de défilement par ID lors de renommages et filtres.
- [ ] Accessibilité : cibles de 48 dp, noms TalkBack, indication textuelle des ignorés et incertains, texte agrandi, portrait/paysage, clavier ouvert, largeur d'overlay minimale existante.

- [ ] Rendre et inspecter des captures Android réelles : trois intervenants, prénom long, homonymes, voix ignorée, correction active, erreur, téléchargement. Joindre les images au rapport Markdown ; corriger les débordements avant T7.

API et présentation retenues pour T6 : le panneau expose une méthode synchrone de rendu `render(document, images, status)`, `flushFocusedEdit()`, `captureAnchor()`, `endEditing()` et `dispose()`. Les callbacks `onAcquireWindow` et `onReleaseWindow` délèguent à l'hôte les flags de focus de l'overlay. `endEditing()` recueille d'abord le champ courant puis termine l'édition avant de rendre le focus ; le panneau ne possède pas de WindowManager. Ce branchement est nécessaire à `OverlayTranscriptEditor`, dont le toucher doit pouvoir acquérir une fenêtre initialement non focalisable. Le contrôleur applicatif appellera `flushFocusedEdit` avant toute mutation ASR ou action de filtre. L'ancre de capture contient `turnId` et un offset dans le corps sérialisé (les marqueurs d'images sont comptés), avec une conversion testée depuis le curseur de l'éditeur graphique. Sans tour, le contrôleur demande le tour documentaire réservé de T5d. Aucun widget ne possède de handle natif ou n'écrit dans les notes.

Le bandeau comporte l'intitulé « Réunion », un accès « Intervenants » avec leur nombre et un menu d'actions ; un statut bref indique l'écoute, la pause, le chargement, la fin ou l'erreur. Les commandes d'enregistrement sont transmises au propriétaire du panneau. Le contrat comprend `START`, `PAUSE`, `RESUME`, `FINISH`, `CANCEL` et `CANCEL_DOWNLOAD` ; les états distinguent aussi `READY`, `PAUSING`, `FINALIZING` et `CLOSING`. Prêt permet de démarrer, le chargement peut être annulé, et les transitions de pause/fin/fermeture ne proposent pas une reprise prématurée. Une note terminée propose « Nouvelle réunion », sans prétendre reprendre sa mémoire vocale perdue. Les profils sont accessibles dans une liste compacte défilante : nom et numéro restent distincts pour les homonymes, « Ignoré » est écrit explicitement. Le texte utilise des lignes de prise de parole séparant l'étiquette tactile et le corps éditable. Le menu ouvre aussi « Passages à vérifier ». À 240 × 160 dp, conserver la zone de texte et les commandes essentielles ; placer les actions secondaires dans le menu et les listes dans une présentation fournie par l'hôte, plutôt que comprimer les cibles tactiles. Le propriétaire fournit une interface de présentation des choix et de saisie de prénom : le panneau ne crée pas de fenêtre Activity ni de fenêtre overlay implicite. Cette interface `MeetingPanelDialogHost` expose `showChoices(request, onChoice)` et `showTextInput(request, onSubmit)` ; les requêtes portent titre, choix identifiés et texte initial si nécessaire. `MeetingPanelActions` transmet les commandes métier. L'accès « Intervenants (n) » ouvre les huit profils au plus par `showChoices`, chacun avec nom, numéro et mention Ignoré si nécessaire ; sélectionner un profil ouvre Renommer ou Ignorer/Réafficher. Aucune View Android n'est injectée dans ce contrat et aucune classe de liste de participants inutilisée n'est créée. Un retour de dialogue cible toujours l'ID initial du profil ou du tour et ne publie rien après `dispose`.

Les retours du panneau sont des commandes identifiées : édition d'un tour, attribution à un ID de profil ou à l'inconnu, renommage, filtre, commande de session et action d'image. Un `TextWatcher` publie les modifications manuelles sans reboucler sur le rendu ; `flushFocusedEdit` récupère aussi une composition en cours. Le rendu met à jour les lignes par diff synchrone et restaure les spans IME originaux, leur sélection et le focus. Les essais passent par le vrai contrôleur du panneau et le reducer, avec un faux hôte de dialogues ; les snapshots simulés servent uniquement aux captures de mise en page. Un hôte d'aperçu peut être ajouté aux sources debug pour les captures instrumentées, clairement désigné comme fixture et absent des sources release.

T7 — Brancher le cycle de vie de l'application

Modifier : `OverlayService.kt`, `PersistencePrefs.kt`, `MainActivity.kt`, `OnboardingActivity.kt` aux points d'entrée/sortie du mode ; `LocalFormatEngine.kt` pour la réservation décrite ci-dessous lorsque son prototype est actif ; `DictationDraftStore.kt` seulement si la liaison au brouillon l'exige. **Créer :** `meeting/MeetingRecordingController.kt` pour coordonner moteur/document/panneau. Généraliser au besoin `RecordingStartupTransaction` pour recevoir un type de session indépendant de Dictée, en conservant ses tests de rollback et ses règles de démarrage ; ne pas introduire une fausse session Dictée pour contourner son type. Les deux workers précédents cessent leurs mutations avant cette intégration.

Exclusivité des modèles : le choix Réunion interdit les nouveaux chargements automatiques de Dictée et de nettoyage. Un chargement Dictée déjà commencé doit terminer puis être libéré avant l'ouverture du modèle Réunion ; sa publication tardive est rejetée par un jeton de mode. Si le prototype de nettoyage local est activé, son moteur partagé doit fournir une réservation asynchrone : refuser les nouvelles demandes de nettoyage/benchmark, attendre la fin du travail natif déjà engagé, puis décharger sur son propre worker. La réservation reste détenue jusqu'à la fermeture confirmée de Réunion. L'annulation pendant cette attente n'allume pas le micro et libère la réservation une fois la fermeture sûre obtenue. Le retour à Dictée réautorise le chargement seulement après cette barrière. Tester l'ordre avec de faux moteurs bloquants, sans attendre sur le thread principal ni fermer un modèle depuis un autre thread pendant son inférence.

Première installation : l'assistant propose aussi le choix Dictée/Réunion. Son étape de modèle dépend du mode : il ne doit pas exiger le téléchargement du modèle Handy de Dictée pour démarrer une réunion. Les permissions micro et overlay restent requises ; l'accessibilité reste facultative en Réunion pour les notes et devient nécessaire seulement aux captures d'écran. Réutiliser le même téléchargement Réunion dans l'assistant et l'accueil. Les vérifications de fichiers volumineux restent hors du thread principal. Tester une installation vierge du prototype et le retour à Dictée.

- [ ] Tests rouges : Dictée démarre uniquement le moteur existant ; Réunion démarre une session à huit voix ; passage de mode pendant RECORDING refusé ; pause/reprise conserve le handle et les profils ; nouvel enregistrement après fin reçoit un nouveau sessionId.
- [ ] Capturer le mode au début du run. Réutiliser `RecordingStartupTransaction`, les permissions et règles FGS existantes ; modèle prêt avant démarrage du micro. Le téléchargement n'arme jamais le micro tout seul.
- [ ] Alimenter la file Réunion depuis le lecteur AudioRecord sans accumulation du `ByteArrayOutputStream` de toute la réunion. PCM libéré après consommation ; métriques du retard sur la même horloge monotone. Le chemin batch de Dictée garde sa capture actuelle.
- [ ] Pause : arrêter la capture puis drainer les blocs déjà reçus, conserver le stream ; reprise sans `finish`. Fin : drainer, finaliser, sauvegarder puis détruire. Annulation : invalider les callbacks, fermer le worker, appliquer la politique de brouillon existante sans injecter un résultat tardif.
- [ ] En Réunion, sauvegarde en note privilégiée ; aucune injection automatique de dizaines de prises de parole dans une autre app à la fin. Export/copie restent des actions explicites. Le texte visible et l'export suivent exactement les mêmes filtres.
- [ ] Désactiver le formatage LLM et les commandes de transformation globale incompatibles avec une réunion active. Garder les éditions et actions images existantes ; les images s'ancrent au document, indépendamment du locuteur et de son filtre.
- [ ] Fusionner les changements d'images concurrents par analyse de diff, sans copier l'ancien `OverlayService.kt` au-dessus du dépôt principal. Tests médias/positions/dossiers/exports requis après cette intégration.
- [ ] Vérifier en instrumentation : pause, annulation pendant chargement, retour arrière, fermeture overlay, changement d'orientation, passage en arrière-plan et destruction du service. Aucun callback ne ressuscite le panneau d'une réunion fermée.

T7a — Réservation du moteur de nettoyage partagé

Lot complexe, limité à `LocalFormatEngine.kt`, au coordinateur et types de réservation dans `LocalFormatMeetingReservation.kt`, et à leurs tests. Ce fichier sépare l'admission et la durée de vie du jeton des opérations natives du core. La méthode de classe `reserveForMeeting(context)` renvoie une demande dont la complétion asynchrone fournit une réservation libérable de façon idempotente. La demande bloque immédiatement l'admission de nouvelles inférences, préparations de benchmark et demandes de préchargement, y compris depuis un nouveau propriétaire du moteur partagé. Cette interdiction est distincte de `GemmaRuntimeAdmission` : le retour d'un watchdog ne peut pas lever une réservation Réunion.

Le worker Gemma traite une barrière après le travail natif déjà commencé ; les tâches encore en attente revérifient la réservation avant de charger un moteur ou de créer une conversation. La barrière décharge le moteur puis résout la demande uniquement si la fermeture est confirmée. L'UI n'attend jamais de façon bloquante. Une annulation marque la demande comme abandonnée, mais ne libère pas prématurément l'interdiction : la barrière termine d'abord la fermeture sûre, puis rend la place. Le code appelant n'annule pas directement une `CompletableFuture` pour signifier cette opération.

API retenue : `reserveForMeeting(context): LocalFormatMeetingReservationRequest`, avec `completion: CompletableFuture<LocalFormatMeetingReservation>` et `cancel()` ; la réservation possède `close()` idempotent. La complétion d'une demande annulée attend elle aussi la barrière sûre. Si la fermeture échoue, cet échec prévaut sur l'annulation et conserve l'interdiction. Après attribution de la réservation, `cancel()` sur la demande ne révoque pas le jeton déjà remis : seul son propriétaire peut le libérer après la fermeture sûre de Réunion. Si la complétion arrive alors que le demandeur a abandonné son démarrage, celui-ci récupère le jeton et le ferme sans ouvrir de session. Cette distinction évite qu'une annulation tardive réautorise le nettoyage pendant une réunion déjà lancée. Les tests injectent le même type de worker FIFO et les opérations de fermeture utilisés par le core ; les gardes d'admission, de début du travail et de début de conversation appartiennent à cette frontière réelle.

Les échecs de destruction jusque-là absorbés par `runCatching` doivent être rendus observables au coordinateur : fermeture de conversation, moteur ou instance d'initialisation échouée. En cas d'incertitude, retenir un état indisponible, échouer la réservation avec un message constant sans cause native et ne pas autoriser le chargement d'un autre gros modèle. Les références ne sont pas réutilisées après une fermeture incertaine. La garde de fermeture mémorise définitivement cet échec et refuse les nettoyages natifs suivants : ne pas réessayer la fermeture de la même référence, ni fermer le moteur parent après une fermeture de conversation incertaine. Cette règle doit aussi couvrir un échec antérieur à la demande de réservation. Les parcours Dictée normaux conservent leurs résultats et deadlines ; aucune modification du modèle, du prompt ou des paramètres de génération.

Tests avec un worker et des opérations natives simulées : demande pendant chargement et génération bloqués, retour d'un watchdog, tâches en attente, nouveau propriétaire pendant réservation, annulation en attente, doubles libérations,

deux demandes concurrentes, échec de fermeture et absence d'inférence après échec. Les tests doivent contrôler l'ordre des appels et les futures, pas se limiter à une lecture de booléen.

T7b — Choix de mode et admission d'un enregistrement

Ajouter une préférence `TranscriptionMode` (Dictée par défaut) et un coordinateur partagé dans le processus, consulté par l'accueil, l'assistant et le service. Le coordinateur conserve une génération de mode et admet au plus un run. Changer de mode est une commande atomique refusée tant qu'un run charge, enregistre, est en pause, finalise ou attend sa fermeture. Le run reçoit un jeton libérable seulement après arrêt effectif du lecteur et fin sûre de sa session native ; une fermeture incertaine maintient l'interdiction. En Réunion, cette fin comprend la fermeture du moteur dédié. En Dictée, le moteur résident peut rester chargé au repos comme actuellement ; sa résidence est suivie séparément et doit être close avant tout chargement Réunion. Ne pas imposer un rechargement du modèle à chaque dictée pour satisfaire le verrou de run. Les écrans ne modifient pas chacun la préférence indépendamment du coordinateur.

Un préchargement Dictée sans enregistrement ne détient pas ce jeton. Il capture cependant la génération de mode ; si le mode change pendant son chargement, il ne publie pas son moteur et le libère sur son worker. Le démarrage Réunion attend la fin de cette opération avant son propre chargement. Un refus de publication ne termine pas le ticket de chargement : il reste comptabilisé jusqu'à la fermeture sûre du moteur rejeté sur son worker. Une fermeture bloquée après ce refus doit donc maintenir la future Réunion en attente ; une fermeture incertaine la fait échouer et verrouille les nouvelles admissions. La fermeture du moteur Dictée doit aussi conserver la trace d'un échec antérieur : `ResidentEngine` dans `TranscriptionEngine.kt` efface actuellement sa référence avant de fermer. Ajouter un état d'échec persistant qui interdit toute nouvelle ouverture et fait échouer les fermetures ultérieures au lieu de rapporter un faux succès sur une référence vide. Tester cette séquence avec un Closeable qui échoue, sans changer le chemin normal de remplacement. Toute fermeture native incertaine, y compris celle d'un préchargement au repos ou pendant la destruction du service, est également signalée au coordinateur partagé : recréer un service dans le même processus ne suffit pas à effacer cette interdiction. Le coordinateur refuse alors toute nouvelle admission Dictée ou Réunion jusqu'au redémarrage du processus ; il ne possède aucune référence vers le service détruit. Ce signal partagé couvre également le moteur de nettoyage de T7a ; sa défaillance ne doit pas rester invisible au prochain chargement ASR. La notification du signal sort de la garde de destruction avant d'appeler des observateurs, pour ne pas exécuter de callback d'interface sous un verrou de ressource native. Tester une fermeture en échec suivie de la création d'un nouveau propriétaire dans le même processus. Les notifications du coordinateur sont exécutées hors verrou ; les vues les reçoivent sur le thread principal et se désabonnent à leur destruction. La mort du processus libère les objets natifs mais pas le document enregistré ; au prochain lancement, aucune restauration ne redémarre le micro automatiquement.

Tester un départ simultané avec un changement de mode, une pause qui conserve le verrou, un chargement Dictée tardif, l'annulation avant ouverture du micro et une libération idempotente. Ce coordinateur est indépendant des fenêtres Android ; il ne garde aucune Activity ni Service. Le jeton de run porte aussi l'identité de son coordinateur, afin que deux compteurs locaux identiques ne rendent pas un jeton étranger valide. `isCurrentRun(lease)` vérifie cette identité, l'ID, le mode, la génération et l'absence de poison ; il sert notamment à l'adaptateur du lease Réunion. L'issue terminale de la readiness est choisie sous le même verrou que l'état, puis sa future est terminée hors verrou. Une issue déjà accordée n'est pas réécrite après coup ; le consommateur revalide toujours le jeton avant d'ouvrir le micro. Le service reste propriétaire du FGS, des permissions et des ressources AudioRecord.

T7c — Captures et images rattachées au document Réunion

Lot complexe, séparé de l'intégration finale de l'overlay. Étendre `PendingNoteCapture` dans `NoteImageStore.kt` avec une ancre Réunion optionnelle contenant `sessionId`, `turnId` et un offset UTF-16 dans le corps sérialisé. Les anciennes réservations restent lisibles et suivent leur chemin Dictée. `beginMeetingBatch` enregistre cette ancre et l'identifiant de note égal à la session, avec les mêmes limites, validation et persistance que les séries existantes. Une capture Réunion est routée d'après cette métadonnée avant toute livraison plate, même si le mode sélectionné a changé après redémarrage. Une ancre incomplète ou invalide ne doit jamais devenir silencieusement une capture Dictée.

Une métadonnée Réunion invalide reste représentée comme une réservation en erreur, par un indicateur dérivé `meetingAnchorInvalid` ; elle ne devient ni une réservation plate ni une absence de réservation. La lecture conserve les données persistées et bloque leur remplacement par une nouvelle série. Les opérations d'ajout ou de livraison la refusent ; seule une annulation explicite peut l'effacer. Le sérialiseur ne transforme pas cet état invalide en une nouvelle réservation sans ancre. Lire les IDs d'ancre comme des chaînes réelles, sans la conversion implicite de `JSONObject.getString`. Pour les écritures et l'effacement de la réservation Réunion, un `commit()` refusé doit restaurer la valeur précédente en mémoire aussi bien que signaler l'échec : les préférences peuvent avoir changé en mémoire avant de retourner false. Tester notamment un effacement refusé suivi d'un nouveau départ ; ce départ ne doit pas écraser la réservation encore récupérable.

Avant d'ouvrir la capture, recueillir la saisie focalisée, capturer l'ancre, puis protéger le corps ciblé via une retouche identique à son texte courant afin que l'ASR ne remplace pas son identité. Sans tour éditable sélectionné, utiliser le tour

documentaire réservé de T5d ; ne jamais convertir un offset de projection filtrée en offset dans des paroles cachées. Mettre en pause la capture audio, attendre son point de sauvegarde T4f puis enregistrer le document dans le brouillon et dans sa note. Créer ensuite la réservation persistante et ouvrir la caméra ou la barre de captures. Toute erreur préalable garde le panneau et le texte accessibles, sans lancer une caméra dont l'ancre n'est pas durable.

À l'acceptation, un composant dédié prépare l'ajout depuis le document courant de cette session, ou son brouillon restauré, ou sa note si aucun brouillon correspondant n'existe. Il déduplique les pièces jointes par ID, contrôle les numéros et la limite globale de dix images, et ajoute seulement les marqueurs encore absents à l'ancre enregistrée, dans l'ordre de capture. Si les nouvelles images dépassent la place restante dans la limite globale, refuser la livraison avec une erreur explicite et conserver toute la réservation ; ne pas tronquer silencieusement les captures avant de l'effacer. Un offset dépassant un corps raccourci est borné à sa longueur, puis ramené à une frontière de caractère s'il coupe une paire UTF-16 de substitution, par exemple un emoji. Si le tour manque, conserver les images dans un tour documentaire visible et signaler que leur emplacement a dû être rétabli. La projection de ce tour documentaire réservé (utterance zéro, sans attribution manuelle) ne porte pas d'étiquette vocale ni de commande d'attribution ; son corps reste éditable. Une photo prise avant la première parole ne doit pas créer un faux « Intervenant à confirmer » dans le panneau ou l'export. Les images déjà rattachées gardent leur position lors d'une répétition de livraison ; aucun doublon de marqueur ou de pièce jointe n'est créé. Toute action opère sur le corps sous-jacent, jamais sur une projection dont le filtre a retiré les paroles.

Ordre durable : fixer d'abord dans la réservation acceptée la correspondance entre chaque ID de capture et son numéro d'image résolu, puis appliquer l'ajout au document en mémoire, attendre le flush du brouillon contenant les marqueurs, puis publier la note structurée avec ses pièces jointes et son document. Ajouter à la réservation `meetingImageNumbersResolved` (faux par défaut pour les anciennes réservations). `resolveMeetingBatchImages(id, images)` conserve exactement les IDs et l'ordre du lot, valide l'unicité des numéros, puis persiste la liste normalisée et cet indicateur avant toute modification du document. Une réservation déjà résolue ne renumérote pas une seconde fois ses images. À la récupération, ses images résolues complètent les pièces connues par ID ; un numéro attribué à un autre ID provoque une erreur récupérable plutôt qu'une substitution. Ainsi, un arrêt après le flush du brouillon mais avant publication de la note retrouve les mêmes marqueurs, même si la première livraison avait dû résoudre une collision de numéros. Tester les arrêts avant/après la persistance de cette correspondance et entre brouillon et note, puis une nouvelle instance de livraison. Le rebase pendant une écriture conserve cette correspondance et l'ordre de capture, y compris lorsque des IDs déjà rattachés alternent avec de nouvelles images. Pour un même ID, refuser un numéro durable contradictoire ; conserver les métadonnées courantes compatibles. Il ne remet pas d'anciennes pièces explicitement supprimées ; distinguer une liste courante vide d'une source courante indisponible. Borner une tentative de livraison à trois flushs si les retouches se poursuivent : au-delà, conserver la réservation et proposer Réessayer plutôt que boucler ou publier un état dépassé. La réservation n'est effacée qu'après succès des deux enregistrements. Un arrêt entre ces étapes laisse la réservation et les fichiers récupérables ; la tentative suivante réconcilie les IDs et les marqueurs. Un échec de note ou de nettoyage conserve la série et affiche une action Réessayer. Les mutations de document restent sur le thread principal ; les opérations d'écriture suivent leur propriétaire et leurs complétions sont revérifiées par sessionId. Les sauvegardes ordinaires ultérieures retrouvent les pièces jointes par cet identifiant, sans remplacer une liste récente par une ancienne copie.

Le composant ne crée pas une seconde instance de `TranscriptNotes` : son cache mutable est actuellement possédé par le service sur le thread principal. Les lectures et publications injectées respectent ce propriétaire, sans envoyer ce même cache sur le worker du brouillon. Une complétion de livraison ne remplace pas le document courant par un snapshot plus ancien si l'utilisateur a continué à l'éditer pendant l'écriture. Pour une réunion vivante, appliquer les corps modifiés à son éditeur existant et créer au besoin son tour documentaire ; ne pas reconstruire cet éditeur avec `restore`, qui désactive volontairement l'ASR des documents restaurés.

Annuler une série non acceptée retire uniquement ses fichiers temporaires et sa réservation. Si une réservation acceptée a ensuite une métadonnée Réunion invalide, son annulation explicite retire la réservation mais conserve les images : une note peut déjà les référencer après une publication réussie suivie d'un effacement interrompu. Réafficher le panneau ne réinjecte pas le texte plat. Une reprise automatique après la capture n'est possible que pour le même run vivant, précédemment en écoute et encore en pause ; une récupération après mort du processus ne redémarre jamais le micro. Conserver la copie dans Photos et ses erreurs telles que prévues par le parcours média, sans transformer une réunion en note plate en cas d'échec. Les actions déplacer/supprimer une image modifient ses marqueurs dans les corps, puis enregistrent le document ; le filtre des voix ne supprime aucune pièce jointe. Ces modifications s'expriment en retouches des seuls corps concernés ; le texte reconnu brut reste inchangé et les tours sans marqueur concerné ne reçoivent pas une protection artificielle. Après suppression explicite, retirer les fichiers privés de l'image seulement après sauvegarde durable du document et de la liste de pièces jointes, afin qu'une panne disque ne laisse pas une note enregistrée pointant vers un fichier déjà effacé.

Tests du composant et du stockage : ancienne réservation sans ancre, ancre invalide refusée, capture avant la première parole, capture sur passage renommé/ignoré, publication interrompue avant ou après chaque étape, double livraison, échec du commit, dix images, déplacement entre deux tours, retour de caméra après annulation ou changement de

session, récupération sans reprise du micro. L'overlay ne contient que l'adaptation de ces commandes ; les décisions de rattachement et l'ordre de publication sont testables séparément.

T7d — Contrôleur de réunion et adaptation finale de l'overlay

Transition vers une nouvelle réunion. Admettre au plus une opération START_NEW, identifiée par la génération d'ouverture, la session et le contrôleur précédent. Refuser une commande périmée pendant un run actif ou une transition. Recueillir puis terminer l'édition avant de présenter l'état de transfert ; aucun champ du document détruit ne reste éditable pendant destroy/clear/claim. Sauvegarder le dernier document dans sa note seulement s'il contient du texte sous-jacent ou des pièces jointes, même si le filtre masque toutes les voix. Une réunion vide ne crée pas une note vide. Revalider chaque retour asynchrone avant de remplacer le panneau ou démarrer le prochain run. Le nettoyage nécessaire peut finir après destruction de l'interface, sans la recréer. En cas d'échec, conserver les données et une reprise explicite.

Actions documentaires accessibles. Compléter le menu « Actions de la réunion » par « Prendre une photo », « Ajouter une capture », « Copier la transcription » et « Exporter la note ». Ajouter `MeetingPanelDocumentAction` (PHOTO, SCREENSHOT, COPY, EXPORT) et un callback `documentAction(action, anchor)` à `MeetingPanelActions`, avec valeur par défaut sans effet pour les anciens hôtes. Le panneau présente ces quatre choix dans les phases stables READY, LISTENING, DELAYED, PAUSED, FINISHED, MODEL_UNAVAILABLE et ERROR ; il les retire pendant chargement, téléchargement, pause, finalisation et fermeture en cours. L'hôte revalide toujours l'état à réception. Une copie ou un export vide ne crée aucun fichier ni note vide.

Avant d'ouvrir ce menu, recueillir l'édition focalisée puis son ancre sérialisée ; transmettre cette ancre initiale aux actions documentaires après le contrôle existant de session et d'ID de choix. La vérifier encore dans l'hôte et appliquer le repli T7c si le tour n'existe plus. Le menu ne ferme pas lui-même le champ et ne crée aucune fenêtre. L'hôte utilise ses vrais chemins de photo/capture, en mettant d'abord la réunion en pause et en attendant le checkpoint avant de réserver les médias. Copie et export utilisent le document courant après flush du champ et du brouillon, puis sa projection structurée. Conserver les actions propres aux images.

L'entrée « Passages à vérifier » reste distincte : remplacer son toast provisoire par une présentation consultable du texte reconnu et du texte conservé pour les passages signalés. Elle reste disponible lorsque le passage appartient à une voix masquée, conformément à T6 ; elle ne modifie pas le filtre de transcription. Tester les nouveaux choix, leur ancre avant dialogue, l'annulation et les retours après changement de session ou destruction. L'extension du panneau constitue un incrément borné séparé ; le worker d'intégration conserve seul le raccordement dans OverlayService.

Placement du panneau et des poignées. Le panneau Réunion occupe toute la largeur sous la barre de déplacement de l'hôte, sans marge inférieure supplémentaire : 240 × 112 dp dans la fenêtre minimale existante. Sa propre rangée de commandes garde cette largeur. Réserver les gouttières latérales au seul RecyclerView des paroles, 36 dp dans la fenêtre réduite et 12 dp lorsqu'elle est agrandie ; elles ne retirent aucune hauteur au texte. Placer les poignées sous les deux rangées de commandes, dans ces bandes, avec une hauteur bornée par l'espace disponible. Aucun bouton ni champ éditable ne doit se trouver sous une poignée. Conserver les dimensions enregistrées et la géométrie de collision de l'overlay. Vérifier la mesure minimale et les transitions réduit/agrandi, puis inspecter le rendu Android.

Exécuter deux incréments testables : T7d.1 livre le contrôleur de réunion avec ses frontières injectées et ses tests de cycle de vie ; T7d.2 adapte ensuite les vrais points d'entrée du service et des Activities, après disponibilité de T7a/T7b/T7c. Le worker qui termine T7c prend T7d.1 ; le worker du panneau garde seul T7d.2. Pendant l'attente d'une vérification T7c, les seuls tests et squelettes compilables de T7d.1 peuvent être préparés dans de nouveaux fichiers indépendants ; les fichiers images restent figés et leur validation reste requise avant raccordement. Après validation de T7b, il peut préparer les sélecteurs de l'accueil et de l'assistant pendant le travail sur le contrôleur, sans anticiper son contrat audio. Le raccordement final d'OverlayService attend la validation de ces frontières. Le contrôleur peut avancer avec des faux ports pendant ces lots, mais ces faux ports ne valident pas le raccordement applicatif. Un seul propriétaire Luna modifie `OverlayService.kt`, `MainActivity.kt` et `OnboardingActivity.kt` lors de cette intégration. `MeetingRecordingController` porte le document, le writer, les commandes du panneau et le cycle de session. Il expose un état immuable et des commandes testables ; le service reste propriétaire de la fenêtre, du FGS et de l'adaptateur AudioRecord. Le contrôleur reçoit des frontières injectables de micro, disponibilité des modèles, exclusivité native, publication de notes et dispatch principal. Une instance appartient à un seul document/session ; sa création distingue explicitement une nouvelle réunion enregistrable d'un document restauré sans ASR. Nouvelle réunion sauvegarde et détache l'ancien contrôleur avant d'en créer un autre. L'arrêt avec attente du lecteur est une future ou une opération de worker, jamais un join sur le thread principal. Une disponibilité Missing/Downloading n'est pas une attente qui lancerait automatiquement le micro après téléchargement : une nouvelle commande de démarrage reste nécessaire. Aucune classe de vue ne charge de modèle ni ne décide de la reprise audio.

États explicites : document consultable sans session, préparation, écoute, mise en pause, pause, finalisation, fermeture et erreur. Le statut de sauvegarde est indépendant : une panne disque conserve l'édition et propose de réessayer. Démarrer réserve le run du coordinateur, interdit les préchargements Dictée, attend leur fermeture et la réservation du nettoyage si nécessaire, puis ouvre MeetingEngine. Seul un `onReady` encore courant permet au service de créer et démarrer le micro.

Revalider les permissions et le droit FGS à ce moment. Chaque continuation remise au thread principal vérifie son identité de run, la génération de mode et l'état du service. Le lease de runtime injecté expose `isCurrentAndUsable()` : le contrôleur le revalide après sa remise, à `onReady` et avant chaque reprise du micro. L'adaptateur vérifie le jeton de run, la génération et le poison global ; un lease invalidé pendant le chargement ne peut pas allumer le micro. Le host conserve également sa vérification de permissions et de droit FGS juste avant AudioRecord.

Le lecteur fournit de petits blocs PCM16 mono 16 kHz à la session et ne conserve pas l'audio complet. Un bloc refusé pendant l'écoute produit un arrêt visible et une finalisation de ce qui a déjà été accepté ; ne pas retenter en boucle ou continuer avec un trou audio caché. Une pause arrête le lecteur, attend sa sortie et sa libération, puis le checkpoint du moteur et le flush du brouillon. Le checkpoint garantit que les callbacks natifs ont été émis, pas que leurs messages UI ont déjà été appliqués. Poster systématiquement sa continuation dans la même file principale FIFO que les hypothèses, y compris lorsque sa future est déjà terminée et que l'appelant est le thread principal ; ne pas exécuter alors le flush immédiatement. Appliquer la même règle après `closed` pour enregistrer les dernières hypothèses de finalisation. Tester une hypothèse déjà postée mais pas encore traitée lorsque checkpoint/closed est observé terminé. La reprise ouvre un nouveau recorder avec la même session native. Terminer suit la même sortie de lecteur, appelle `finish` une seule fois, accepte les derniers événements valides, marque le document terminé, vide le writer et publie la note. L'exclusivité native est libérée seulement après fermeture sûre ; une panne de sauvegarde ne permet pas de perdre le document, mais n'impose pas de garder inutilement les modèles en RAM. Si `MeetingSession.closed` échoue parce que la fermeture native n'est pas confirmée, le port de runtime signale l'incertitude au coordinateur global et garde le jeton de run ainsi que la réservation du nettoyage. Aucun timeout ne les libère ensuite. Cette interdiction dure jusqu'à la mort du processus, tandis que le document reste éditable et sauvegardable. Une erreur d'inférence suivie d'une fermeture confirmée permet, elle, la libération normale. Le signal global et cette rétention restent effectifs après destruction des vues. Annuler ou détruire le service invalide les futurs retours UI, préserve les mots et retouches déjà reçus, arrête le lecteur et ferme le natif sur son worker. Les continuations de libération des ressources et de mise à jour de l'exclusivité ne dépendent pas des callbacks UI supprimés par `main.removeCallbacksAndMessages(null)` : elles terminent même après destruction de l'overlay. Séparer ces finalisations thread-safe des publications de vues et de notes possédées par le thread principal. Tester une fermeture lente qui se termine après destruction, puis un nouveau propriétaire admis seulement après cette fin sûre.

Le brouillon utilise un propriétaire unique dans le processus, indépendant du verrou des modèles. Deux instances de service ne doivent pas créer deux writers concurrents sur le même AtomicFile : son verrou actuel est limité à une instance. Ajouter une frontière de transfert du brouillon, indexée par son chemin privé et sans référence d'Activity/Service. Une nouvelle instance attend la fin de la fermeture du writer précédent avant de lire ou de modifier ce fichier, y compris si l'ancien document était seulement consulté sans session native. Le dernier snapshot immuable est remis à cette frontière avant destruction. Le registre expose une requête `claim(path)` avec future et annulation explicite. Le claim accordé fournit writer, relais d'erreurs, document récupéré, erreur de sauvegarde, `rememberSnapshot(document)` et `relinquish(snapshot)` asynchrone. Les factories du contrôleur reçoivent ce claim et actualisent son snapshot après mutation. Une fabrique de claim interne peut isoler les tests unitaires du contrôleur ; les essais de transfert doivent utiliser le vrai registre, ses claims et le writer. La lecture initiale du fichier et le transfert des écritures passent par la frontière d'E/S, sans bloquer le thread principal. Si la dernière écriture échoue, attendre la fin de cette tentative puis retirer définitivement l'ancien writer avant d'accorder le claim suivant. Étendre MeetingDraftWriter avec ce chemin explicite de retraite, admis seulement après close et sans écriture active : il arrête son scheduler et interdit toute nouvelle mutation ou tentative de flush/close, sans rapporter un faux succès de sauvegarde. La fermeture ordinaire conserve son comportement de nouvelle tentative tant qu'aucun transfert n'est demandé. Le nouveau propriétaire reçoit le snapshot en mémoire et l'erreur, puis crée un nouveau writer qui accepte les retouches et peut réessayer. Détacher le relais d'erreurs de l'ancienne UI avant destruction ; le registre ne conserve pas de callback de Service ou d'Activity. Une mort réelle du processus ne garantit évidemment que le dernier état écrit sur disque. Les callbacks UI tardifs ne peuvent plus écrire après transfert. Cette barrière ne garde aucun modèle natif chargé : sa libération reste liée à la fermeture native, tandis que l'admission d'un nouveau document attend la fin du transfert documentaire. Tester deux services successifs avec une dernière écriture bloquée, puis réussie ou refusée ; l'ancien writer ne doit ni écraser le nouveau document ni supprimer son fichier temporaire pendant une lecture.

Toutes les mutations documentaires sont ordonnées sur le thread principal. Avant chaque hypothèse ou changement de filtre, le contrôleur recueille le champ focalisé, applique la mutation puis rend le snapshot synchrone. Une édition issue du TextWatcher met à jour le document et programme sa sauvegarde sans déclencher une boucle de rendu du même champ. Noms, attributions et filtre ne redémarrent pas le micro. Le contrôleur expose aussi `ensureDocumentTurn()` et `flushDraft()` pour les images, la copie et les exports. Le premier utilise le tour documentaire stable de son éditeur vivant et conserve le suivi de mutation habituel. Le second recueille le champ focalisé, mémorise le snapshot et attend le writer, sans terminer la session ni publier une note par lui-même. Les corps produits par T7c sont appliqués par les commandes d'édition à cette même instance, avec création du tour documentaire si nécessaire ; le service ne reçoit pas l'éditeur mutable et ne restaure pas un document vivant pour y insérer des images. Les diagnostics de retard utilisent le temps audio et le temps monotone, en distinguant file en attente et calcul en cours ; aucun texte privé n'est journalisé.

Une note structurée reconnue s'ouvre toujours dans le panneau Réunion en lecture/édition, sans accepter d'ASR. La fabrique de restauration accepte aussi explicitement un `MeetingDocument` provenant d'une note, avec le claim de brouillon et les ports : un fichier de brouillon absent ne doit pas empêcher cette ouverture. Conserver l'entrée actuelle de récupération du claim en la faisant déléguer vers cette fabrique explicite. Tester une note connue avec claim Absent : identités, noms, retouches et images restent stables, le document est terminé, son édition/flush fonctionne et aucun modèle ni micro n'est ouvert. L'hôte sauvegarde et détache le document précédent avant de choisir cette source ; cette fabrique ne décide pas du parcours de bibliothèque. Une structure inconnue reste consultable par son texte de repli et ne peut pas être aplatie par Enregistrer. Une réunion restaurée après mort du processus indique que l'enregistrement est terminé/interrompu ; enregistrer de nouvelles voix crée une nouvelle session et une nouvelle note, après sauvegarde de l'ancienne, avec les profils réinitialisés. La sélection du mode depuis l'accueil ou l'assistant passe par T7b ; l'overlay suit cette sélection lorsqu'il est au repos et refuse les bascules occupées. Ne pas confondre le type du document consulté avec une session native encore vivante.

Lorsqu'une note Réunion est affichée sans session native, la commande « Nouvelle réunion » sauvegarde cette note, demande le mode Réunion au coordinateur puis crée un autre document. La pastille suit le mode sélectionné : si un autre type de document est encore ouvert, le sauvegarder et le détacher avant de préparer le prochain enregistrement. Une panne de sauvegarde refuse ce départ et laisse le document accessible. Aucun choix de mode ne convertit une note structurée en texte plat, et aucune simple consultation de note ne démarre le micro.

Adapter explicitement les points existants : départ/pause/reprise/arrêt de la pastille, masquage du panneau, réouverture de note, sauvegarde, titre/dossier, copie/export, actions caméra et capture, erreurs, retour de capture et destruction. Le masque du panneau conserve l'enregistrement et expose une description adaptée à Réunion. Copier ou exporter vide d'abord le champ focalisé et le brouillon, puis utilise la projection de la note structurée. Les transformations LLM et l'insertion automatique de fin restent absentes de cette voie. Les tailles et frontières anti-collision de l'overlay sont conservées. Les gestes Dictée gardent leur comportement ; les commandes Réunion de la pastille suivent le contrat spécifique T7d.2 ci-dessous.

Tests du contrôleur avec faux ponts : annulation à chaque étape de préparation, modèle indisponible, retour tardif du chargement Dictée, micro non initialisé ou start refusé, saturation, pause/reprise, fin depuis pause, fermeture lente ou incertaine, callback reçu après destruction, panne de sauvegarde et reprise, ancienne note restaurée puis nouveau départ. Les essais d'intégration exercent les vrais points d'entrée du service et des Activities ; une suite de faux widgets seuls ne valide pas ce raccordement.

Contenu documentaire sans paroles. Le contrôle « ne pas créer de note vide » tient compte des pièces jointes, détenues par l'hôte et absentes du modèle de document vocal. Étendre le port de publication avec `hasAttachments(sessionId): Boolean`, méthode par défaut fausse qui conserve l'usage des faux publishers simples. Le contrôleur consulte ce fait sur le thread principal, en plus des corps sous-jacents non vides ; l'hôte interroge les pièces courantes de cette session dans son unique cache de notes. Une réunion avec des images reste publiable après effacement de tous ses marqueurs textuels, et son état final doit être sauvegardé. Une erreur de lecture de ce fait est une erreur récupérable de sauvegarde, pas la preuve d'un document vide. Tester le document réellement vide et le document sans paroles mais avec pièce jointe ; masquer toutes les voix ne rend jamais le document vide.

Lease pendant la pause Dictée. La pause existante conserve sa session ASR pour reprendre la même dictée. Elle conserve donc aussi son lease de run ; ne pas terminer ou recréer cette session uniquement pour permettre un changement de mode. Le choix d'un autre mode est refusé jusqu'à fin ou annulation, arrêt du lecteur et fermeture native sûre, avec « Terminez la dictée avant de changer de mode. ». Le mode Réunion suit le même principe pendant sa pause. Distinguer toutefois une session native vivante d'une simple note affichée sous l'ancien état UI `PAUSED` : sans run ni session native, cet état ne bloque pas le changement de mode. La sauvegarde et le détachement du document précédent restent obligatoires avant le nouvel enregistrement. Une fermeture incertaine conserve l'interdiction jusqu'à la mort du processus.

T7d.1 — Effacement ordonné du brouillon connu

Complément rédigé par Astra après lecture du registre : `MeetingDraftStore.clear()` seul ne suffit pas, car le registre garde le dernier snapshot en mémoire. L'hôte utilise une commande asynchrone `MeetingDraftOwnership.clear(path, expectedSessionId)`, ordonnée dans la même file que les claims. Elle attend la libération du propriétaire et la fin du writer précédent, puis vérifie l'identité du document avant tout effacement. Elle ne peut pas effacer une autre session arrivée entre-temps ni une structure inconnue ou invalide. Un état réellement Absent réussit sans modifier de note. L'opération ne possède aucun modèle ni vue et ne dépend pas du Handler du service.

L'effacement du fichier passe par `MeetingDraftStore.clear()` sur l'exécuteur d'E/S ; son succès remet le snapshot, le document récupéré et l'erreur en état Absent, puis admet le prochain claim. Si l'effacement échoue, conserver le snapshot en mémoire, rendre l'erreur et laisser une récupération ou nouvelle tentative explicite possible. La comparaison de session et la mutation du cache sont sérialisées avec les transferts. Aucun callback ni achèvement de future ne se produit sous le

verrou. Les suppressions directes du brouillon depuis l'overlay sont interdites : elles pourraient restaurer une note que l'utilisateur vient de fermer ou supprimer.

L'hôte appelle cette commande après sauvegarde durable et détachement du document précédent, ou dans le parcours de suppression déjà demandé par l'utilisateur, avant d'ouvrir un autre document. Une erreur bloque cette transition et laisse le texte récupérable. Tester l'écriture précédente bloquée, le claim suivant qui reçoit Absent après succès, le refus d'une autre session et d'un payload inconnu, l'absence de résurrection par l'ancien writer et l'échec d'effacement conservant le snapshot. Lot complexe, propriété séparée sur le registre et ses tests ; deux revues Astra avant raccordement.

T7d.2 — Contrat complémentaire des gestes et de la spirale

Adaptateurs Android avant le raccordement final. Pour garder les opérations audio et les réservations hors du gros fichier OverlayService, réaliser un lot borné `meeting/MeetingNativeAdmission.kt` et `meeting/MeetingAudioRecord.kt`, avec leurs tests. Il peut suivre la validation du contrôleur pendant le travail sur les Activities. Le premier implémente `MeetingNativeReservationPort` avec le coordinateur T7b, une fermeture asynchrone du moteur résident fournie par l'hôte et la réservation du formateur T7a lorsqu'elle est applicable. Le run Réunion est réservé avant toute ouverture ; sa disponibilité et la fermeture du formateur constituent deux barrières. Une annulation avant leur remise attend la fermeture sûre, puis libère les ressources acquises sans ouvrir le micro. Une fermeture incertaine signale le poison global et conserve les réservations jusqu'à la mort du processus. Le lease combine identité courante du run et interdiction locale après libération ou poison. Aucun callback ni future n'est complété sous un verrou de cet adaptateur. Les continuations de nettoyage ne dépendent pas du Handler des vues. Tester les deux ordres de fin des barrières, annulation précoce et tardive, refus de réservation, validité du lease et maintien de l'exclusivité après poison.

Le second adaptateur implémente `MeetingMicrophoneFactoryPort`/`MeetingMicrophonePort` : AudioRecord mono PCM16 à 16 kHz, blocs courts et bornés, boucle de lecture dédiée, aucune accumulation de l'audio complet. Une vérification injectée depuis le service revalide permission micro, service vivant et armement FGS immédiatement avant création/démarrage. Il expose une frontière de recorder remplaçable en test ; les essais ne construisent pas de fausse session Dictée. `start` ne rapporte un succès qu'après validation du recorder et lancement de son lecteur. Les chemins de refus arrêtent et libèrent leurs ressources. `stopAndJoin` est idempotent, arrête le lecteur et termine sa future après sa sortie et la libération du recorder, sans join sur le thread principal. Un bloc refusé ou une erreur de lecture arrête ce lecteur ; le contrôleur conserve la décision de finaliser et d'afficher l'erreur. Tester démarrage refusé, permission retirée, bloc refusé, arrêt pendant lecture bloquée, commandes répétées et libération après retour du lecteur. Ces adaptateurs ne possèdent ni fenêtre, ni note, ni préférence de format ; le worker UI conserve seul OverlayService. Leur preuve réelle de micro attend les essais applicatifs, séparés des faux recorders.

Incrément des Activities, rédigé par Astra. Après T7b et avant le raccordement audio, le propriétaire UI ajoute un sélecteur explicite Dictée/Réunion sur l'accueil, puis une page de réglages Réunion accessible à côté de Dictée. L'assistant présente le même choix avant ses étapes. Les deux écrans consultent `TranscriptionModeCoordinator.process` et demandent `changeMode` ; le refus laisse le choix courant visible et indique « Terminer l'enregistrement avant de changer de mode ». Un état de fermeture incertaine utilise le message d'indisponibilité du moteur. Les commandes mesurent au moins 48 dp et restent lisibles avec police agrandie. La page Réunion montre la langue globale, le suivi de huit voix au maximum et l'installation des deux modèles, environ 849 Mo, avec les états absent, vérification, progression, prêt et erreur. Elle partage `MeetingModelStore`, ses commandes de téléchargement/annulation et ses notifications avec l'assistant ; aucun téléchargement ne démarre à la simple ouverture de page. Les notifications du store sont remises au thread principal et détachées à la destruction. La progression actualise ses vues sans reconstruire toute la page ni déplacer son défilement. Aucun écran ne charge le runtime natif. L'assistant en Réunion n'exige ni le modèle Dictée ni l'accessibilité ; une ligne facultative explique que l'accessibilité sert aux captures d'écran. L'armement du service reste une action depuis l'Activity visible et n'ouvre pas le micro. Le branchement final du service doit supprimer ses préchargements Dictée en mode Réunion avant validation applicative de cet incrément. Tests des vrais écrans : mode initial, sélection et refus pendant un run, format Dictée conservé, étape du bon modèle, téléchargement sans démarrage audio, callback tardif après destruction et retour à Dictée. Les fixtures utilisent un store de test injecté, sans réseau ni gros modèles ; elles ne valident pas un enregistrement.

Origine et auteur : demande explicite de l'utilisateur du 24 septembre, spécifiée et planifiée par Astra. Ce contrat complète l'intégration T7d.2 ; il ne change pas les composants T6 actuellement gelés pour leur validation. Difficulté complexe, deux revues Astra sur le même état final.

Propriétaire : le même worker Luna qui possède l'adaptation finale d'OverlayService. Ajouter à son périmètre `CursiveWaveView.kt`, un composant borné `MeetingPillInteraction.kt` si nécessaire pour isoler la table de commandes, et leurs tests JVM/Robolectric/instrumentés. Réutiliser les reconnaisseurs de gestes existants ; ne pas ajouter une seconde couche de listeners concurrents. Le contrôleur T7d.1 expose uniquement son état et ses commandes ; il ne dépend pas des gestes ou de la géométrie du dessin. Après les preuves natives T6, le worker peut préparer puis vérifier le dessin circulaire et la table pure des gestes dans ces fichiers bornés pendant T7b/T7d.1. Ce travail indépendant ne touche

pas encore OverlayService ni les sélecteurs partagés. Le branchement aux vrais états attend leur validation ; une animation de fixture ne prouve pas la commande du micro.

Menu vers le haut. Au repos, le menu existant des formats reçoit une ligne de sélection « Dictée / Réunion » en tête, avec deux cibles de 48 dp minimum et leur état sélectionné. Le glissement court révèle ce menu sans changer de choix ; le mode se choisit par un appui explicite sur sa commande. La sélection prolongée des formats garde les seuils et le comportement de `FormatSwipeGesture` : le premier mouvement ouvre, un second mouvement intentionnel sélectionne. La ligne de modes ne fait pas partie des indices de formats et ne peut donc pas être sélectionnée accidentellement par un ancien index. En Dictée, garder la liste et le format courant. En Réunion, masquer cette liste inapplicable, montrer le mode courant et un accès explicite au panneau. La sélection de Réunion conserve le format mémorisé de Dictée. En cours de réunion, le menu peut rappeler le mode mais refuse toute bascule via T7b, y compris en pause. Les raccourcis vers le haut d'une dictée déjà en cours restent inchangés.

Signature de la table pure. `MeetingPillInteraction` utilise ses propres phases d'interface : READY, PREPARING, LISTENING, PAUSING, PAUSED, FINALIZING, CLOSING, FINISHED, RESTORED, MODEL_UNAVAILABLE et ERROR. Il reçoit un geste TAP, SWIPE_DOWN ou SWIPE_UP, ainsi que les faits `hasDocumentContent` et `dialogOpen`, et retourne une intention sans effet : NONE, START_NEW, PAUSE, RESUME, PROMPT_FINISH, SAVE_OPEN_NOTE, SHOW_PAUSE_HINT, OPEN_MODE_MENU ou SHOW_MEETING_PANEL. L'adaptateur traduit l'état du contrôleur ; aucune dépendance vers sa phase documentaire ambiguë n'est imposée à la table. Un dialogue ouvert et les phases transitoires neutralisent les nouvelles commandes. Une note finie/restaurée reçoit START_NEW au tap et SAVE_OPEN_NOTE en descente si elle contient des données. Une indisponibilité ou erreur ouvre le panneau pour ses actions adaptées. En pause, la proposition de fin reste possible même si aucun texte n'a encore été publié : la finalisation peut fournir les derniers mots avant le contrôle de document vide. Le mode visuel expose `setMeetingMode` et `setMeetingCaptureActive` ; les méthodes de géométrie et de snapshots restent internes aux tests.

Appui et geste vers le bas. Appliquer la table d'états de la spec depuis un état capturé puis revalidé à l'exécution. Réunion n'emprunte pas la logique de tap/double tap qui arrête ou insère une dictée. Le premier appui depuis l'écoute demande la pause ; un second appui pendant PAUSING est ignoré, sans reprise mise en attente. Un appui depuis PAUSED reprend la même session native. Le glissement vers le bas devient admissible pour la proposition d'enregistrement lorsque PAUSED est effectif, après arrêt du lecteur et checkpoint ; le mouvement initial n'arrête ni n'enregistre à lui seul. En écoute, l'aide indique « Mettre en pause pour enregistrer ». Depuis la pause, l'aide indique « ↓ Enregistrer ». Pour une note terminée/restaurée, cette commande enregistre ses retouches sans rouvrir de modèle. Un document réellement vide, sans corps ni pièce jointe, ne crée pas une note vide ; un filtre masquant toutes les voix ne rend pas vide le document sous-jacent.

Proposition de fin. Recueillir le champ focalisé avant d'ouvrir le dialogue et utiliser les libellés exacts de la spec. Le dialogue capture sessionId/runId et se ferme ou refuse son action si ce contexte a changé. N'avoir qu'un dialogue de fin à la fois ; neutraliser le tap de reprise pendant qu'il est ouvert. Fermer ou choisir « Continuer la réunion » conserve PAUSED et les profils. « Enregistrer et terminer » appelle la commande de fin existante du contrôleur une seule fois ; le libellé du panneau peut être adapté à cette intention sans modifier son contrat. Le succès durable conserve le mode Réunion sélectionné, le dessin fixe et la note consultable ; le prochain appui ouvre une nouvelle session. Une panne de sauvegarde garde les données et propose Réessayer ; la fermeture native sûre ne dépend pas de la réussite disque. Les gestes répétés pendant préparation, pause en cours, finalisation ou fermeture ne programment aucune action différée. Aucun export ni insertion dans une autre application n'est automatique.

Dessin et animation. Ajouter un mode visuel Réunion explicite à `CursiveWaveView`, indépendant du mode de marque et du preset compact Dictée. Conserver exactement les chemins et paramètres de la Dictée et du logo. Dessiner une couronne fermée de six boucles cursives reliées, avec leurs croisements et de légères variations fixes, dans le plus grand carré centré qui tient dans la pastille actuelle. La couronne doit rester reconnaissable à petite taille, avec un centre ouvert, le trait actuel d'environ 2,3 dp et une marge incluant son épaisseur. Ni pétales pleins, ni rayons, ni arc de chargement. La rotation globale, cible initiale d'un tour en 3,2 secondes, crée l'effet spiralé ; le niveau micro peut moduler légèrement l'ouverture des boucles, sans agrandir l'enveloppe ni traduire tout le dessin. Les six boucles n'encodent pas le nombre de participants. Astra valide le résultat sur capture native et séquence animée, puis fixe une éventuelle correction bornée de géométrie.

État visuel et coût. La rotation suit le fait que le lecteur capture réellement, pas seulement un état générique « occupé ». Elle continue pendant un silence lorsque le micro est ouvert. Au démarrage refusé ou au chargement, elle reste fixe avec le statut existant distinct. À la pause, figer l'angle dès l'arrêt effectif du micro, même si des calculs ou une sauvegarde restent à terminer ; la reprise poursuit cet angle sans saut. Mesurer le temps écoulé avec l'horloge monotone, borner les grandes reprises de frame, recycler Path/Paint, ne pas allouer à chaque frame. Arrêter les callbacks hors fenêtre, écran non visible, au repos et à la destruction ; la reprise visuelle ne déclenche jamais le micro. Si Android désactive les animations, utiliser la couronne fixe et les indications d'écoute/pause accessibles. Les thèmes manuel clair/sombre utilisent ThemeTokens comme aujourd'hui. Le mode et l'état restent lisibles dans la description accessible et le panneau ; le mouvement ne constitue pas le seul indicateur.

TDD et preuves. Écrire d'abord les cas comportementaux rouges : appui écoute→pause, double appui pendant PAUSING sans reprise, pause→reprise sans nouveau profil, descente avant pause sans fin, descente en pause ouvrant une seule proposition, fermeture sans reprise, confirmation unique, session changée avant confirmation, échec disque et nouvelle tentative, choix Réunion sans changement du format Dictée, ouverture courte du menu sans sélection, glissement annulé ou multi-touch sans commande, déplacement long conservé. Vérifier ensuite le rendu et le scheduler : dessin circulaire contenu, aucune translation horizontale, angle immobile au repos/chargement/pause, rotation en capture, continuité pause/reprise, réglage sans animations, detach/reattach sans callbacks doublés. Les tests existants de formats, tap Dictée, déplacement et onde restent verts. Le contrôleur conserve ses propres tests de fin depuis pause et de fermeture sûre.

Audit visuel final. Captures Android des deux modes, menu avec modes et formats, proposition de fin, note enregistrée, thèmes clair/sombre, taille minimale et police agrandie. Ajouter une séquence courte de la spirale en écoute puis en pause, provenant de la vue native, et des images à plusieurs phases dans l'artefact Markdown. Distinguer fixture de dessin, gestes réellement exercés dans OverlayService et micro réellement ouvert. Aucun rendu navigateur ne vaut une preuve Android. Le panneau T6 gelé peut être approuvé séparément avant cet ajout ; la validation applicative T8 couvre ensuite l'ensemble.

T8 — Validation de livraison et limites mesurées

Créer/mettre à jour : `docs/research/meeting-mode/validation.md`, `docs/research/meeting-mode/README.md`, notices natives/modèles, `CLAUDE.md` Session Log. Astra rédige les conclusions ; Luna fournit mesures et captures.

Installation de test isolée : prévoir `-PmeetingPrototype=true`, avec applicationId `com.uhama.whisperpin.meetingtest`, libellé « DictAI Réunion — test » et versionName distincte. Le namespace Kotlin et les classes JNI restent inchangés. Les autorités du FileProvider suivent déjà `${applicationId}` et les actions de capture sont envoyées à un composant explicite ; les vérifier dans l'APK. Les essais émulateur utilisent cette variante et ses propres fichiers privés, pour conserver l'installation DictAI existante. Aucun changement de version ou de données de cette dernière n'est nécessaire.

- [ ] Exécuter sous JDK21 `./gradlew :app:testDebugUnitTest :app:assembleDebug` et les tests instrumentés pertinents. Vérifier `git diff --check` ; fournir nombre réel de tests, échecs et identité APK.
- [ ] Scénario humain fonctionnel : Personne 1 → Sophie, Personne 2 → Karim, retour de Sophie, correction mardi → jeudi, Personne 3 ignorée puis réaffichée, voix inconnue, pause/reprise, export, réouverture. Vérifier le texte exporté caractère par caractère sur ce scénario.
- [ ] Rejouer le témoin T1 contre la version finale avec réglages figés : ASR seul et ASR+diarisation, au moins trois essais. Publier p50/p95 délai des mots et des tags, retard maximal, mémoire PSS, temps de chargement. Séparer les paroles reconnues de l'exactitude d'attribution ; ne pas réduire la qualité à un nombre de tags distincts.
- [ ] Effectuer une réunion d'au moins 30 minutes sur Poco F7 physique quand connecté : retours de voix, huit participants, bruit, chevauchements, écran éteint, chauffe, consommation, RAM, éditions. En l'absence du Poco, statut explicite « APK expérimental, validation physique restante » ; aucune promesse de direct utilisable sur téléphone.
- [ ] Vérifier le mode Dictée avec ses propres modèles : pas de chargement diarisation, pas de remplacement de GGML, édition et injection conservées. Contrôler aussi images, dossiers, export PDF et pastille.
- [ ] Astra fige les empreintes du diff/artéfacts puis réalise les deux revues requises, corrige via tâches Luna bornées si nécessaire et relance uniquement les validations affectées.
- [ ] Identifier l'APK par applicationId/versionCode/versionName/ABI/SHA-256 et chemin exact. Aucune publication distante ni commit/push sans instruction explicite. Un build terminé ne vaut pas une installation ni un test sur Poco.

Frontière des fixtures du service. Astra fixe un `OverlayService.MeetingTestOverrides` interne par instance : `draftOwnership`, `draftFile`, `modelStore`, puis uniquement les ports `modelAvailability`, `reservation`, `sessionFactory` et `microphoneFactory`. Les adaptateurs du thread principal, du champ focalisé et de publication des notes restent toujours ceux du vrai service ; injecter tout `MeetingRecordingPorts` contournerait précisément l'édition et la sauvegarde à vérifier. `installMeetingTestOverrides(overrides)` s'appelle sur le thread principal avant toute création de panneau, claim, contrôleur ou run, et refuse une installation tardive. Aucun Intent, préférence ou écran ne l'expose. Le retrait accompagne la destruction, sans échange de ports pendant une session. Le service garde son unique cache `TranscriptNotes` réel ; les fixtures créent des identités de notes uniques et ne nettoient que leurs propres ressources. La fixture instrumentée refuse un package autre que le prototype ; les tests JVM ordinaires restent exécutables avec la commande Gradle par défaut.

Réouverture documentaire et mode. Depuis Dictée au repos, `openNote` sélectionne Réunion via le coordinateur avant d'ouvrir une note structurée connue avec le contrôleur restauré. Aucun modèle, microphone ni run n'est ouvert ; les modèles absents laissent l'édition disponible. Refuser l'ouverture d'une autre note tant qu'un run réel existe, même si `captureActive` est faux : préparation, pause et fermeture conservent ce verrou. Préserver le document précédent en cas de refus. Un contenu de version inconnue garde sa présentation protégée et son texte original ; il n'est ni aplati ni migré

implicitement. Un poison sans run n'autorise aucune capture et ne justifie jamais de supprimer le document : la consultation documentaire reste indépendante de la disponibilité du moteur.

Pour l'instrumentation Android, une factory interne à usage unique dans le companion fournit les overrides avant la restauration de `onCreate`. Elle reçoit l'instance du service, est consommée puis remise à null avant invocation, et passe son résultat au même installateur gardé. La fixture peut ainsi retenir l'instance réelle sans réflexion sur les API Android cachées. Son setter refuse d'écraser une factory déjà pendante ; un clear explicite la retire dans le finally si aucun service ne démarre. Aucune factory n'est installée dans le parcours produit. Ce mécanisme ne contourne ni les permissions ni l'armement du FGS et ne retarde pas la restauration normale du brouillon.

Une première fixture Android utilisera des ports natifs simulés pour exercer les vrais gestes, dialogues, focus, panneau, writer et notes. Elle ne devra pas être présentée comme une capture réelle. La preuve micro et celle du moteur JNI conservent leurs essais distincts ; le raccordement final doit préciser quelles frontières chaque scénario traverse.

Scénario de la première fixture des gestes réels. Démarrer le service du prototype depuis une Activity visible, avec permission micro et overlay accordées extérieurement puis restaurées après le runner. Une factory de test à usage unique fournit un répertoire de brouillon isolé, deux petits fichiers vérifiés par un catalogue de test, et les seules frontières natives simulées. La réservation utilise néanmoins le vrai coordinateur pour conserver le refus de changer de mode pendant une session. Aucun vrai modèle ni son n'est chargé. Les événements tactiles traversent la vraie fenêtre et son listener ; les captures attendent des frames présentées et vérifient l'état visible.

Le scénario ouvre le menu vers le haut et le panneau sans démarrer. Un tap démarre, puis trois hypothèses contrôlées créent Personne 1, Personne 2 et le retour de Personne 1. Renommer la première Sophie depuis le dialogue, corriger « La séance aura lieu mardi. » en « La séance aura lieu jeudi. » dans le vrai champ, puis livrer une révision tardive : le prénom et la retouche restent stables. Pendant l'écoute, une descente affiche l'aide sans finaliser. Un tap lance la pause ; une barrière d'arrêt contrôlée permet de vérifier qu'un second tap pendant PAUSING ne programme aucune reprise. Après PAUSED, ouvrir une seule confirmation, la fermer par Continuer et vérifier que le micro reste arrêté. Reprendre par tap dans la même session et avec les mêmes profils, mettre en pause puis confirmer Enregistrer et terminer. Vérifier la note structurée durable avec les prénoms et retouches attendus. Le tap suivant crée une autre identité de session avec des profils vides, après préservation de la note précédente. Conserver des captures du menu, des tours renommés, de la confirmation et de la note finie. Les erreurs de disque, la récupération de processus et les médias gardent leurs tests d'intégration dédiés ; cette première fixture ne les remplace pas.

Contrôle du service avec ses adaptateurs réels

Smoke du service avec ses adaptateurs réels. Après la fixture de gestes simulés, exécuter séparément le prototype avec `meetingTestOverrides == null`, le store partagé de production et les deux modèles épinglés déjà disponibles localement. La factory à usage unique peut seulement retenir l'instance du service et retourner null ; elle ne remplace aucune frontière. Ne télécharger aucun poids pour cet essai. Préparer un paquet de modèles à identité unique, vérifié par taille et SHA, et ne nettoyer que ce paquet après fermeture sûre. Exiger un brouillon Réunion standard absent au départ ; s'il existe, ne pas l'écraser ni le supprimer pour le test.

Démarrer depuis l'Activity visible avec les permissions accordées puis restaurées extérieurement. Une vraie commande de démarrage doit traverser l'admission, les fermetures résidentes, le moteur JNI et AudioRecord. Attendre LISTENING et le FGS microphone, garder quelques secondes de capture, mettre l'Activity en arrière-plan tout en conservant l'overlay, puis mettre en pause et reprendre la même session native. Terminer depuis la pause et attendre les fermetures effectives ainsi que la libération du coordinateur. Enregistrer les phases et l'identité de la session, sans conserver de son. Si le document final est vide, vérifier qu'aucune note vide n'est créée ; sinon vérifier la publication structurée et nettoyer seulement son ID connu. Cet essai sur émulateur valide le raccordement par défaut et son cycle de vie, sans prouver une voix audible, l'exactitude de la diarisation, la latence sur Poco ou une réunion longue. Garder distincts les journaux des gestes simulés et ceux des adaptateurs réels.

Contrôle de l'export Android

Complément de vérification rédigé par Astra : produire avec les vrais exporteurs Android un PDF, un HTML autonome et le texte partagé d'une réunion synthétique. Sophie annonce « La séance aura lieu mardi. », retouché en « La séance aura lieu jeudi. » ; Karim répond « Je prépare les documents. » ; Sophie reprend « Nous vérifierons ensemble le matériel. ». Une troisième personne ignorée porte « BRUIT À MASQUER [[Image 1]] ». Un passage attribué provisoirement à cette voix, mais non confirmé, conserve « La salle reste à confirmer. » sous « Intervenant à confirmer ». Un tour documentaire sans locuteur contient « Annexe de la réunion [[Image 2]] ». Les deux images synthétiques portent « Plan de salle » et « Liste du matériel ». L'image du passage ignoré reste exportée.

Le champ texte plat de la note contient volontairement « CACHE OBSOLÈTE À EXCLURE » : aucun export ne doit le reprendre. Comparer le texte partagé à une attente littérale indépendante de la projection ; vérifier les prénoms actuels, la correction jeudi, l'incertitude conservée, les deux images dans l'ordre et l'absence des trois chaînes obsolètes/masquées.

Exporter le PDF natif, rendre toutes ses pages à 150 dpi et les faire inspecter par Astra. La fixture utilise un stockage privé isolé et aucun modèle, microphone ou réseau ; elle valide l'export des données structurées, pas leur production par la reconnaissance.

Protocole de mesure T8

Employer le même audio figé et les mêmes paramètres CPU pour six passages cadencés, dans l'ordre ASR seul / ASR avec diarisation / ASR avec diarisation / ASR seul / ASR seul / ASR avec diarisation. Rapporter chaque passage séparément avant d'agréger les trois essais de chaque variante. Ne pas attribuer une différence au seul coût de la diarisation si les conditions de cache, d'échauffement ou de charge hôte diffèrent. Le temps de préparation inclut chargement et échauffement. L'origine du temps de flux correspond au début de l'acquisition audio simulée, après préparation ; chaque bloc n'est fourni qu'à son échéance de fin, lorsque tous ses échantillons seraient disponibles sur un micro réel. Pour un bloc commençant à l'offset `s` et contenant `n` échantillons à 16 kHz, l'échéance est donc `(s + n) / 16000` seconde après cette origine. Fournir le bloc dès son offset de début avancerait artificiellement les résultats d'une durée de bloc et fausserait le délai des mots.

Pour les délais, compter une seule première apparition par mot repéré dans l'utterance et sa position audio, en conservant les révisions sans les compter comme de nouveaux mots. Distinguer première attribution positive et attribution stable confirmée ; les finales réémises pour stabiliser un locuteur ne multiplient pas le nombre d'observations. La confirmation utilise le vrai seuil `stable_speaker_time()` du stream et un tag positif ; la fin du fichier ne transforme pas arbitrairement tous les tags en attributions stables. Compter séparément les mots positifs mais encore non confirmés. Calculer le délai entre temps monotone de publication et échéance de fin du mot selon l'horloge audio du runtime. Ces délais restent dépendants de ses timestamps estimés ; ils ne mesurent pas à eux seuls l'exactitude des mots ou des voix. Publier la définition, les effectifs et les valeurs p50/p95, ainsi que les cas non attribués en fin de passage.

La comparaison ASR seul/avec diarisation peut utiliser un mode de benchmark de la sonde Android T1, avec les mêmes révisions et réglages natifs que le pont final ; elle est alors identifiée comme mesure du processus de sonde, distincte de la validation JNI et UI de l'APK. Conserver le scénario T1 existant. Ajouter aux traces les identifiants d'utterance et de révision pour rapprocher les premières apparitions des mots finaux sans additionner les hypothèses répétées ; signaler les rapprochements ambigus plutôt que fabriquer leur délai. Le script de coexistence doit sélectionner explicitement les neuf bibliothèques historiques de son manifeste, car le répertoire contient désormais aussi `libdictai_meeting.so`. Ne pas compter ce dixième fichier comme une ancienne bibliothèque.

Mesurer le retard maximal et la mémoire PSS du processus Android réellement testé à cadence de sondage fixée, en précisant si le sondage lui-même affecte l'essai. Conserver la trace brute hors données privées, l'identité des binaires et de l'audio, les paramètres et la commande. Les observations de l'émulateur servent au mécanisme et aux régressions ; seules les mesures du téléphone établissent son aptitude au direct.

Suivi d’exécution

Lot	Avancement	Preuve attendue
Spec et plan	Rédigés par Astra ; enregistrés dans le dépôt	Ces deux fichiers
T0 Référence	Validée : 543 tests initialement ; médias intégrés exactement, 665 tests et build commun verts	Manifeste des 35 fichiers médias et preuves T0b
T1 Runtime Android	Approuvé après deux revues Astra ; français synthétique 1 → 2 → 3 → 1 avant fin	Tags avant EOF avec vrai modèle ; qualité humaine et Poco restent à mesurer
T2 Profils/projection	Approuvé ; extension images et exports approuvée avec T5c	Tests identités, renommage, filtre
T3 Retouches	Approuvé après deux revues ; 33 tests	Raccordement clavier réel encore à valider en T6/T7
T4 Moteur/paquet	File, JNI, session, checkpoint et stockage approuvés ; 6 tests JNI applicatifs réussis	Exclusivité et chargement UI à raccorder en T7
T5 Sauvegarde	Codec, brouillon atomique, notes, éditeur restauré et writer approuvés ; exports Android et rendus PDF/HTML vérifiés	Branchement du service, reprise et publication des retouches à valider
T6 Interface	Panneau approuvé après deux revues ; 33 tests ciblés et 12 captures Android inspectées	Hébergement réel, gestes et ASR à valider en T7d/T8
T7 Intégration	T7a/T7b/T7c, contrôleur, adaptateurs et effacement du brouillon approuvés ; gestes purs et huit captures Activities validés ; raccordement du service en cours	Réunion réelle dans l’application, gestes, sauvegarde et médias
T8 Livraison	Outils approuvés, 18 tests Python verts, six mesures de sonde Android et exports PDF/HTML/TXT vérifiés ; résultats dans validation.md	Validation applicative après T7 ; APK final et essai physique restants

Auto-revue du plan par Astra

- Couverture : modes T4/T7 ; profils et noms T2 ; ignorer/réafficher T2/T6 ; direct T1/T4 ; retouches T3/T6 ; images T5/T7 ; pause/nouvelle session/reprise T5/T7 ; huit voix et incertitude T1/T2 ; erreurs/mémoire T4/T7 ; exports T5 ; mesures T8.
- Décisions encore soumises à preuve, pas à délégation implicite : vitesse Android, consommation et qualité humaine. T1 puis T8 portent explicitement ces validations.
- Frontières de travail : natif, domaine, interface et intégration séparés ; aucune mutation concurrente d’OverlayService pendant l’intégration finale.
- Le développement est autorisé par « Ok vas-y au travail ». La demande ultérieure précise que l’auteur de la spec et du plan doit rester Astra ; elle ne délègue pas leur rédaction et ne suspend pas le développement.